# MIT 18.335J《Introduction to Numerical Methods》学习笔记

> **课程**：18.335J / 6.337J Introduction to Numerical Methods（Spring 2019，MIT 数学系 + EECS） **主讲**：Prof. Steven G. Johnson（课程大纲署名；部分历史讲义幻灯片署名 Per-Olof Persson，见各讲 PDF 首页） **层次**：Graduate（研究生课）；**先修**：18.06 或 18.700 线性代数或同等内容 **教材**：Trefethen & Bau, *Numerical Linear Algebra*, SIAM, 1997（ISBN 9780898713619） **参考**：Heath, *Scientific Computing: An Introductory Survey*；Barrett et al., *Templates for the Solution of Linear Systems*；Bai et al., *Templates for the Solution of Algebraic Eigenvalue Problems* **公开材料**：MIT OCW 课程页 `https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/`（lecture-notes / syllabus / calendar 三页已逐页抓取，PDF 逐份 `pdftotext -layout` 阅读） **本笔记定位**：18.06 教你"分解存在且唯一"，本课教你"浮点里它还算不算数"。重点 = 浮点、稳定性、条件数、复杂度与实现陷阱。 **访问日期**：2026-11（所有外部链接见第 27 章）


1.1 一句话定位
1.2 先修到底要什么
1.3 考核方式（只写大纲明写的）
1.4 讲次地图（39 讲，按 OCW calendar 页原样）
1.5 标注约定（读懂本笔记的钥匙）
1.6 与 18.06 的关系：一张对照表
1.7 五个专题的导航（对应任务要求的专题切分）
2.1 问题：把 $\mathbb{R}$ 换成有限集 $\mathbb{F}$
2.2 机制：IEEE 754 的位布局
2.3 误差模型：一条式子撑起整门课
2.4 复杂度 / 代价：浮点运算不是免费的
2.5 实现陷阱
2.6 本章要点
3.1 问题：$n$ 个数相加，误差能有多大?
3.2 算法（朴素求和）
3.3 误差分析：两个界，一个条件数
3.4 更好的算法与代价
3.5 实现陷阱
3.6 本章要点
4.1 问题：换一个范数，结论会不会崩?
4.2 证明（四步）
4.3 常用范数与诱导矩阵范数
4.4 条件数为什么与范数无关（量级上）
4.5 实现陷阱
4.6 本章要点
5.1 问题：错的是算法还是问题?
5.2 定义与基本不等式
5.3 例子：Hilbert 矩阵有多坏
5.4 条件数不是"行列式小"
5.5 复杂度
5.6 实现陷阱
6.1 问题：$\sqrt{a}$ 怎么算
6.2 算法与收敛证明
6.3 收敛速率：二次收敛的定量形式
6.4 复杂度
6.5 实现陷阱
7.1 问题：把一组向量变成标准正交基
7.2 算法：CGS 与 MGS
7.3 误差分析：CGS 丢正交性，MGS 不丢
7.4 复杂度
7.5 实现陷阱
8.1 问题：要"三角化"，不要"正交化"
8.2 算法：Householder 反射
8.3 误差与稳定性
8.4 复杂度
8.5 Givens 什么时候值得用

8.6 实现陷阱
9.1 问题：一次分解，多次求解
9.2 算法：带部分主元的高斯消元
9.3 为什么必须选主元：一个 2×2 的反例
9.4 增长因子：主元能选多好?
9.5 误差与稳定性
9.6 复杂度
9.7 实现陷阱
10.1 问题：对称正定矩阵的最优分解
10.2 算法
10.3 误差与稳定性
10.4 复杂度
10.5 实现陷阱
11.1 问题
11.2 两条路线
11.3 实测：Hilbert 矩阵上的最小二乘
11.4 更稳的做法与代价阶梯
11.5 实现陷阱
12.1 问题：任何矩阵都是"旋转—拉伸—旋转"
12.2 与特征分解的关系（以及为什么不能直接对 $A$ 求特征值）
12.3 Eckart–Young：低秩逼近的最优性
12.4 复杂度
12.5 实现陷阱
13.1 问题：为什么不能像解线性系统那样"一次做完"
13.2 Schur 分解：迭代的目标形态
13.3 特征值问题的条件数
13.4 算法：Householder 归约到 Hessenberg / 三对角
13.5 误差与稳定性
13.6 复杂度
13.7 实现陷阱
14.1 问题：只要一个特征值/特征向量
14.2 Rayleigh 商：把向量变成特征值的估计
14.3 幂法及其收敛率
14.4 反迭代与 Rayleigh 商迭代
14.5 复杂度
14.6 实现陷阱
15.1 问题：同时求所有特征值
15.2 为什么它等于"同时幂法"
15.3 位移：从"能收敛"到"能用"
15.4 复杂度
15.5 实现陷阱
16.1 问题：$A$ 太大，不能分解
16.2 Arnoldi 算法与 Arnoldi 关系
16.3 Lanczos：对称情形的三项递推
16.4 Ritz 值与"幽灵特征值"
16.5 误差与稳定性
16.6 复杂度
16.7 实现陷阱
17.1 问题：非对称线性系统
17.2 用 Arnoldi 关系把问题降维
17.3 收敛界（以及它为什么几乎没用）
17.4 实测：重启的代价
17.5 复杂度
17.6 实现陷阱
18.1 问题：让 Krylov 方法"看得见"谱
18.2 共轭梯度：三项递推 + A–范数最优
18.3 收敛界
18.4 这个界有多松：一个实测
18.5 预处理子一览
18.6 非对称与不定：CG 的亲戚
18.7 实现陷阱
19.1 问题：内存不够时，怎么"接着算"
19.2 必须保住的性质
19.3 朴素重启为什么失败（讲义的核心推导）
19.4 正确做法：隐式重启
19.5 复杂度
19.6 实现陷阱
20.1 问题：零也是要花钱的
20.2 存储：压缩列（CSC）
20.3 填充（fill-in）：稀疏直接法的核心困难
20.4 排序策略
20.5 直接法的复杂度（讲义原表）
20.6 实现陷阱
21.1 问题：flop 数不能预测时间
21.2 理想缓存模型与缓存复杂度
21.3 讲义给的 25 行实现

21.4 但缓存无关还不够：寄存器也是缓存
21.5 实践数字（讲义平台，2008 年）
21.6 实现陷阱
22.1 问题：$O(N^2)$ 的 DFT
22.2 Cooley–Tukey：$N = N_1 N_2$ 的指标重排
22.3 术语与算法家族
22.4 复杂度与"常数之战"
22.5 实测
22.6 实现陷阱
23.1 问题：$O(1/N^2)$ 还是指数收敛?
23.2 讲义的两个实验
23.3 用 Fourier 余弦级数解释"奇迹"
23.4 从梯形到 Clenshaw–Curtis
23.5 复杂度
23.6 实现陷阱
24.1 问题：为什么数值线性代数课要讲优化
24.2 拟 Newton 与 BFGS
24.3 伴随方法：一次反向传播换全部梯度
24.4 复杂度
24.5 实现陷阱
25.1 与 18.06 线性代数笔记
25.2 与 18.100A 实分析笔记
25.3 与 Stanford EE364a 凸优化笔记
25.4 与 CS336 大模型笔记（数值稳定）
25.5 一张"学完能做什么"的清单
26.1 复习清单（能不看笔记答"是"才算过）
26.2 自测题（10 题，答案在后）
26.3 答案
27.1 抓取与核验方式
27.2 课程页面
27.3 讲义 PDF（27 份，均已下载并转文本阅读）
27.5 本笔记的可靠性分级

# 第 1 章 课程概览：从"精确的代数"到"近似的算术"

## 1.1 一句话定位

18.06 证明 $A=LU$、$A=QR$、$A=U\Sigma V^{\mathsf{T}}$ 存在；18.335 问的是另一串问题：**在 IEEE 双精度的 53 位尾数下，这个分解算出来的东西离真值多远？代价多少个 flop？在缓存上跑得多快？** 用课程自己的话（Lec 1 讲义开篇），数值方法区别于分析与理论计算机科学的地方有三条：

1. 它处理**任意实数**（及其向量空间扩张），结果一般不是整数或有理数，只能用有理数逼近；
2. 除了"解存在且正确"，还关心**时间与内存**（"computer science = math + time = math + money"）；
3. 还关心**精度**：算法本身可能是固有近似的（Newton 法永远到不了精确解），而算术本身也是近似的（$\mathbb{R}$ 被换成了浮点集 $\mathbb{F}$，每次 $+,-,\times,\div$ 都要舍入）。

本笔记的三个贯穿问题，建议每读完一章都回来对一遍：

| 贯穿问题 | 一句话判据 | 主要出现章节 |
| --- | --- | --- |
| **误差从哪来?** | 前向误差 $\lesssim$ 条件数 × 向后误差 | 第 3、5、9、11 章 |
| **算法稳定吗?** | 它是不是某个"邻近问题"（nearby problem）的精确解 | 第 7、8、9、10、13 章 |
| **值得算吗?** | flop 数 × 内存层次，别只数 flop | 第 21、22 章 |

## 1.2 先修到底要什么

大纲写的是 18.06 / 18.700 或同等。按讲义实际用到的内容拆开：

| 先修知识点 | 在哪一讲立刻被用 | 若忘了看哪里 |
| --- | --- | --- |
| 四个基本子空间、正交投影 | Lec 7（SVD）、Lec 9（QR/最小二乘） | 18.06 笔记"正交投影"部分 |
| 特征值、相似、Schur/谱定理 | Lec 14–17 | 18.06 笔记"矩阵分解速查表" |
| 范数、内积、Cauchy–Schwarz | Lec 4（范数等价） | 本笔记第 4 章（含完整证明） |
| 紧性、极值定理、数列收敛 | Lec 4 证明第 4 步、Lec 1 收敛证明 | 18.100A 笔记"紧性/极值定理" |
| 一点编程（Julia/MATLAB/Python） | 每周作业 | 本笔记配套速查文档 |

课程**不要求**事先会 Julia：大纲明确写"little or no prior experience required; you will learn as you go"。OCW 另附 *Introduction to Julia* 与 *Julia & IJulia Cheat-Sheet* 两份 PDF。

## 1.3 考核方式（只写大纲明写的）

| 项目 | 占比 | 说明 |
| --- | --- | --- |
| Problem sets（4 次） | 33% | 大纲："There are four problem sets" |

| 项目 | 占比 | 说明 |
| --- | --- | --- |
| Take-home midterm exam | 33% | 期中为带回家考试，日历标注"due before Lec #25" |
| Final project | 34% | 8–15 页单栏单倍行距综述论文，需含自做数值实验、与至少一个竞争算法的公平比较、完整文献引用 |

> 注意：OCW 大纲只给了这三项的百分比，**没有**给出"迟交扣分""协作细则"以外的量化规则。协作政策原文三条：不得参考往年作业解答；与同学讨论或搜索前必须先自己认真尝试；无论跟谁讨论、读什么，最终解答必须自己独立书写（不得对着别人的答案写）。这些是课程政策原文要点，不是笔者总结的"经验"。

## 1.4 讲次地图（39 讲，按 OCW calendar 页原样）

| 周 | 讲次 | 主题（calendar 页原文的意译） | OCW 是否提供该讲 PDF |
| --- | --- | --- | --- |
| 1 | 1 | 课程概览、Newton 求根 | ✅ lec1 |
| 1 | 2 | 浮点算术 | ✅ lec2 + lec2_supp |
| 2 | 3 | 浮点求和与向后稳定性 | ✅ lec3-1 / lec3-2 |
| 2 | 4 | 向量空间上的范数 | ✅ lec4 |
| 2 | 5 | 条件数 | ❌（仅日历主题） |
| 3 | 6 | 常微分方程数值方法 | ✅ lec6 |
| 3 | 7 | SVD、应用与条件数 | ❌ |
| 3 | 8 | 线性回归与广义 SVD | ❌ |
| 4 | 9 | 用 QR 与 Gram–Schmidt 解正规方程 | ⚠️ 仅 lec9_reading（Gram–Schmidt） |
| 4 | 10 | 修正 Gram–Schmidt 与 Householder QR | ✅ lec10 |
| 4 | 11 | 矩阵运算、缓存与分块 | ✅ lec11 |
| 5 | 12 | 缓存无关算法与空间局部性 | ✅ lec12 |
| 5 | 13 | LU 分解与部分主元 | ❌ |
| 5 | 14 | Cholesky、专用求解器、特征问题与 Schur | ❌ |
| 6 | 15 | 特征值算法：友矩阵、病态性、Hessenberg 归约 | ✅ lec15 |
| 6 | 16 | 幂法与 QR 算法 | ⚠️ lec16_reading（QR Algorithm I/II） |
| 6 | 17 | 带位移 QR 与 Rayleigh 商 | ❌ |

| 周 | 讲次 | 主题（calendar 页原文的意译） | OCW 是否提供该讲 PDF |
|---|---|---|---|
| 7 | 18 | Krylov 方法与 Arnoldi | ❌ |
| 7 | 19 | Arnoldi/Lanczos 的重启 | ✅ lec19 |
| 7 | 20 | GMRES 及其收敛性 | ❌ |
| 8 | 21 | 预处理技术、共轭梯度 | ❌ |
| 8 | 22 | 共轭梯度的收敛性 | ❌ |
| 8 | 23 | 双共轭梯度类算法 | ❌ |
| 9 | 24 | 稀疏直接法 | ✅ lec24 |
| 9 | 25 | 优化问题概览 | ✅ lec25 |
| 9 | 26 | 伴随方法 | ✅ lec26 |
| 10 | 27 | 特征问题与递推关系的伴随/灵敏度 | ✅ lec27 |
| 10 | 28 | 信赖域与 CCSA | ❌ |
| 11 | 29 | Lagrange 对偶 | ❌ |
| 11 | 30 | 拟 Newton 法与 BFGS | ✅ lec30 |
| 11 | 31 | BFGS 更新推导 | ❌ |
| 12 | 32 | 无导数优化的线性/二次近似 | ❌ |
| 12 | 33 | 数值积分与梯形公式的收敛 | ✅ lec33_1 / lec33_2 |
| 12 | 34 | Clenshaw–Curtis 求积 | ❌ |
| 13 | 35 | Chebyshev 逼近 | ❌ |
| 13 | 36 | 带权积分与 Gauss 求积 | ❌ |
| 13 | 37 | 自适应与多维求积 | ❌ |
| 14 | 38 | DFT 与 FFT 算法 | ✅ lec38 |
| 14 | 39 | FFT 算法与 FFTW | ✅ lec39 |

**材料可得性的诚实清单**（这是本笔记最重要的一条元信息）：

- OCW lecture-notes 页实际只挂了 **27 个 PDF**（含 Julia 入门、Julia 速查、ideal-cache 术语表、lec2a 浮点迷思），**不是 39 讲每讲一份**。
- `/pages/assignments/` 在抓取时返回 **HTTP 404 / "Page Not Found"**；课程导航里也**没有** Assignments 或 Exams 栏目。因此本笔记**不引用任何题号、分数、期中试题内容**——它们不在公开材料里。

- 因此凡是没有 PDF 的讲次（第 5、7、8、13、14、17、18、20–23、28–29、31–32、34–37 讲），本笔记的内容来自 **calendar 页的主题名 + Trefethen & Bau 教材对应章节的公共知识**，并在段首用 `〔教材〕` 或 `〔推断〕` 标注，绝不伪装成讲义原文。

## 1.5 标注约定（读懂本笔记的钥匙）

为了让"事实可追溯"和"我的推断"不混在一起，全文用四种行内标记：

| 标记 | 含义 | 例子 |
|---|---|---|
| `〔讲义 LecN〕` | 直接来自该讲 PDF（已下载并转文本阅读） | Householder flop 数 $2mn^2 - \frac{2}{3}n^3$ |
| `〔教材〕` | Trefethen & Bau 或 Heath 的标准结论，课程未给 PDF 但在大纲教材内 | $PA = LU$ 的向后稳定性 |
| `〔实验〕` | 笔者用 NumPy 2.3.5 在写作时实跑得到，附代码或数字 | CGS 的 $\|I - Q^{\mathsf{T}}Q\| \approx 3.07$ |
| `〔推断〕` | 笔者的工程经验/合理外推，未经课程材料确认 | "生产代码里应优先用 bf16 而非 fp16" |

## 1.6 与 18.06 的关系：一张对照表

| 主题 | 18.06 的说法 | 18.335 追加的问题 |
|---|---|---|
| $A = LU$ | 存在条件 = 顺序主子式非零 | 不选主元会怎样？增长因子能到多大？ |
| $A = QR$ | Gram–Schmidt 或 Householder 都能得到 | CGS 在 $\kappa(A) = 10^8$ 时 $Q$ 已完全不正交，而 $R$ 仍然对——为什么？ |
| 最小二乘 | 解正规方程 $A^{\mathsf{T}}A\hat{x} = A^{\mathsf{T}}b$ | $\kappa(A^{\mathsf{T}}A) = \kappa(A)^2$，所以正规方程把可用位数砍半 |
| 特征值 | $\det(A - \lambda I) = 0$ | 特征值问题本身可能是病态的（即便矩阵"看起来正常"） |
| SVD | 存在性 + 几何意义 | 算 SVD 要多少 flop？截断误差怎么估？ |
| 正交矩阵 | $Q^{\mathsf{T}}Q = I$ | 浮点里只有 $\|Q^{\mathsf{T}}Q - I\| = O(\kappa\epsilon)$ |

**一句话**：18.06 的定理全部成立，只是它们成立在 $\mathbb{R}$ 上；18.335 研究它们在 $\mathbb{F}$ 上的"折扣率"，而折扣率由**条件数**给出。

## 1.7 五个专题的导航（对应任务要求的专题切分）

| 专题 | 包含章节 | 一句话主线 |
|---|---|---|
| 浮点专题 | 第 2、3 章 | $\mathbb{F}$ 是有限集 → 每次运算 1 个相对误差 → 累积成 $n\epsilon$ 或 $\sqrt{n}\epsilon$ |
| 线性方程专题 | 第 9、10、18 章 | LU/Cholesky 是直接法（$n^3/3$），CG 是迭代法（每步 $O(\mathrm{nnz})$） |
| 最小二乘与 QR 专题 | 第 7、8、11 章 | 永远不要显式形成 $A^{\mathsf{T}}A$ |
| 特征值与 SVD 专题 | 第 12–15、19 章 | 先归约（Hessenberg/tridiagonal），再迭代（QR/Arnoldi） |
| 稀疏与 Krylov 专题 | 第 16–20 章 | 只存非零、只做 matvec、靠预处理救收敛 |
| FFT 专题 | 第 22 章 | $N = N_1 N_2$ 的指标重排 → $O(N \log N)$ |

# 第 2 章 浮点算术与 IEEE 754

> 对应材料：Lec 2《Floating-Point Arithmetic, the IEEE Standard》（Per-Olof Persson 署名幻灯片）与 Lec 2a《Some myths about floating-point arithmetic》（S. G. Johnson，2013-09-13）。

## 2.1 问题：把 $\mathbb{R}$ 换成有限集 $\mathbb{F}$

浮点数的科学计数法原型是 $\pm 1.602 \times 10^{-19}$：符号、有效数字、基数、指数。二进制下讲义给出的一般形式为〔讲义 Lec 2〕

$$\pm \left( d_0 + d_1 \beta^{-1} + \cdots + d_{p-1} \beta^{-(p-1)} \right) \beta^{e}, \qquad 0 \le d_i < \beta$$

其中 $\beta$ 是基数、$p$ 是精度（有效位数）、指数范围 $[e_{\min}, e_{\max}]$。**关键性质**：相邻浮点数之间的间隙随数值大小**按比例**缩放——这就是为什么误差要用相对误差而不是绝对误差衡量。

机器精度（machine epsilon）的定义与量级〔讲义 Lec 2〕：

$$\epsilon_{\text{machine}} = \tfrac{1}{2}\beta^{1-p} \implies \forall x \in \mathbb{R},\ \exists x' \in \mathbb{F}:\ |x - x'| \le \epsilon_{\text{machine}} |x|$$

对双精度 $p = 53$，故 $\epsilon_{\text{machine}} = 2^{-53} \approx 1.11 \times 10^{-16}$。

> **易混点**：`numpy.finfo(float).eps` 返回的是 $2^{-52} \approx 2.22 \times 10^{-16}$，那是 **1.0 处的间距**（ulp），是 Trefethen 记号的 $2\epsilon$。写误差估计时用哪个必须说清楚，否则会差一倍。〔实验：`np.finfo(float).eps == 2**-52` 为 True〕

## 2.2 机制：IEEE 754 的位布局

| 项目 | 单精度 single | 双精度 double |
|---|---|---|
| 符号 / 指数 / 尾数位 | 1 / 8 / 23 | 1 / 11 / 52 |
| 有效位 $p$ | 24 | 53 |
| 总位数 | 32 | 64 |
| $e_{\max}$ / $e_{\min}$ | $+127$ / $-126$ | $+1023$ / $-1022$ |
| 最小规格化数 | $2^{-126} \approx 10^{-38}$ | $2^{-1022} \approx 10^{-308}$ |
| 最大规格化数 | $2^{127} \approx 10^{38}$ | $2^{1023} \approx 10^{308}$ |
| $\epsilon_{\text{machine}}$ | $2^{-24} \approx 6 \times 10^{-8}$ | $2^{-53} \approx 10^{-16}$ |

上表逐项来自〔讲义 Lec 2〕的 "IEEE Floating Point Data Types" 页。单精度的编码规则（讲义原表）：

- $E = 0, M = 0 \to \pm 0$；$E = 0, M \neq 0 \to$ **非规格化数**（denormalized），值为 $(-1)^S \times 0.M \times 2^{-126}$；
- $0 < E < 255 \to$ 规格化数 $(-1)^S \times 1.M \times 2^{E-127}$；
- $E = 255, M = 0 \to \pm\infty$；$E = 255, M \neq 0 \to$ NaN。

讲义给的解码例子（可直接当自测）：`0 10000001 101000…` $= +1 \cdot 2^{129-127} \cdot 1.101_2 = 6.5$；`0 00000000 100000…` $= 2^{-127}$；`0 00000000 000…001` $= 2^{-149}$（最小非规格化数）。

## 2.3 误差模型：一条式子撑起整门课

若舍入是**正确舍入**（correctly rounded，即舍入到最近，平局取偶 round-to-nearest-even），则对任意浮点数 $x, y$ 与运算 $* \in \{+, -, \times, \div\}$〔讲义 Lec 2〕：

$$x \circledast y = (x * y)(1 + \epsilon), \qquad |\epsilon| \le \epsilon_{\text{machine}}$$

这条式子是后面所有误差分析的唯一公理。它立刻给出两个推论：

1. **整数运算在浮点里是精确的**（只要不超 $2^{53}$），所以"整数比浮点准"是错的〔讲义 Lec 2a〕；
2. 误差是**相对的**，所以"给结果加了一个 $O(\epsilon)$ 的随机噪声"是错的——$1 \oplus 1$ 永远精确等于 $2$，$0 \otimes x$ 永远精确等于 $0$〔讲义 Lec 2a〕。

讲义 2a 列出的"浮点迷思"里，对写代码最有用的四条：

| 迷思 | 事实 |
|---|---|
| 每次运算都加一个 $O(\epsilon)$ 随机误差 | 错；只有不可精确表示的结果才舍入 |
| 计算精度 = 计算准确度 | 错；整数可精确存储，而累积误差可让准确度远低于存储位数 |

| 迷思 | 事实 |
| --- | --- |
| 中间步骤用不到的高精度是浪费 | 错（Kahan）；最终只要 3 位有效数字，中间也可能需要很多位 |
| 二进制浮点对十进制小数"随机"舍入 | 对二进制浮点为真（$0.1$ 不可表示），对十进制浮点为假 |

## 2.4 复杂度 / 代价：浮点运算不是免费的

- 一次双精度加乘在当代 CPU 上约 1–4 个周期（有 FMA 时乘加融合为 1 条指令），但**内存搬运**往往是瓶颈（第 21 章）；
- 非规格化数（subnormal）在部分硬件上触发**慢路径**，可能慢 10–100 倍〔推断：依 CPU 微架构而定，x86 上现代实现多在硬件里处理，ARM 某些核会显著变慢〕；
- 单精度相对双精度：尾数少 29 位，但寄存器吞吐常翻倍（SIMD 通道数翻倍），这是深度学习用 bf16/fp16 的算术动机。

## 2.5 实现陷阱

**陷阱 1：$0.1 + 0.2 \neq 0.3$。** 这不是 bug，是 $0.1$ 在二进制下是无限循环小数。实测〔实验〕：

```
>>> 0.1 + 0.2 - 0.3
5.551115123125783e-17
```

任何"用 `==` 比较浮点"的代码都是错的；比较应写成 `abs(x-y) <= tol*max(1,abs(x),abs(y))`。

**陷阱 2：灾难性抵消（catastrophic cancellation）。** 两个相近数相减，有效位被整体削掉。经典例子是二次方程求根 $x = \dfrac{-b \pm \sqrt{b^2 - 4ac}}{2a}$，取 $a = 1, b = 10^8, c = 1$：

| 公式 | $x_1$ | 相对误差 |
| --- | --- | --- |
| 直接公式（分子相减） | $-7.450580596923828 \times 10^{-9}$ | $\approx 25\%$ |
| 稳定写法 $x_1 = c/q,\ q = -\frac{1}{2}\left(b + \mathrm{sign}(b)\sqrt{b^2 - 4ac}\right)$ | $-1.000000000000000 \times 10^{-8}$ | $\approx 0$ |

〔实验〕：真值 $x_1 = -10^{-8}$；直接公式把 $\sqrt{b^2 - 4ac} \approx b$ 的 8 位有效数字全部抵消掉了。**判据**：当 $b^2 \gg 4ac$ 时必须用稳定写法。

**陷阱 3：$1 - \cos x$ 在 $x \to 0$ 时的下溢。** $x = 10^{-8}$ 时直接算 $(1 - \cos x)/x^2$ 得 `0.0`（$\cos x$ 舍入成 $1.0$），而 $2\sin^2(x/2)/x^2$ 得 $0.5$〔实验〕。同类还有 $\log(1 + x)$（用 `log1p`）、$e^x - 1$（用 `expm1`）、$\sqrt{1 + x} - 1$。

**陷阱 4：把"非规格化"当成不会发生。** 若不清零（flush-to-zero）地处理非规格化数，$\|x - y\|$ 可能在 $x \neq y$ 时算成 $0$，于是 `if x != y: z = 1/(x-y)` 这类代码会炸——这正是讲义里引入 gradual underflow 的动机：保证 $x = y \iff x - y = 0$〔讲义 Lec 2〕。

**陷阱 5：把 `float32` 当"省一半内存的 float64"用。** 它的 $\epsilon = 2^{-24} \approx 6 \times 10^{-8}$，累加 $10^6$ 项时最坏相对误差可达 $10^6 \times 6 \times 10^{-8} = 6\%$〔推断：由 $n\epsilon$ 界直接代入〕；深度学习因此规定"参数/激活 bf16，**累加与优化器状态 fp32**"（见第 25 章）。

## 2.6 本章要点

1. $\epsilon_{\text{machine}} = 2^{-53}$（双精度，Trefethen 记号），每步运算相对误差至多 1 个 $\epsilon$；IEEE 的四类特殊值（$\pm 0, \pm\infty, \text{NaN}$、非规格化）不是装饰，它们保证了"比较—减法—除法"这条链的自洽；
2. 抵消是**唯一的**"凭空丢位"机制；条件数（第 5 章）是它的定量版本。

---

# 第 3 章 舍入误差、求和与向后稳定性

> 对应材料：Lec 3a《Notes on the accuracy of naive summation》（2019–02–08）与 Lec 3b《Backwards stability of recursive summation》（2012 创建，2015–02–19 更新），均为 S. G. Johnson 讲义。

## 3.1 问题：$n$ 个数相加，误差能有多大?

求和是"最笨"的算法，却恰好能干净地展示三个核心概念：**前向误差界、条件数、向后稳定性**。设精确和为 $f(x) = \sum_{i=1}^{n} x_i$，浮点顺序求和（"recursive summation"，讲义特意指出这与是否用递归实现无关，只与运算顺序有关）为 $\tilde{f}(x) = \tilde{s}_n$，其中 $\tilde{s}_k = \tilde{s}_{k-1} \oplus x_k$。

## 3.2 算法（朴素求和）

```
sum = 0
for i = 1 to n
    sum = sum + x_i
return sum
```

## 3.3 误差分析：两个界，一个条件数

**前向界**〔讲义 Lec 3a，含归纳法证明〕：

$$|\tilde{f}(x) - f(x)| \;\le\; n\,\epsilon_{\text{machine}} \sum_{i=1}^{n} |x_i| + O(\epsilon_{\text{machine}}^2)$$

证明的关键引理：存在 $|\epsilon_k| \le \epsilon$ 使

$$\tilde{f}(x) = \sum_{i=1}^{n} x_i \prod_{k=i}^{n} (1+\epsilon_k), \qquad \epsilon_1 = 0$$

把连乘展开成 $1+\delta_i$，$|\delta_i| \le n\epsilon + O(\epsilon^2)$，再用三角不等式即得。于是**相对**误差为

$$\frac{|\tilde{f}(x) - f(x)|}{|f(x)|} \le n\,\epsilon_{\text{machine}} \cdot \underbrace{\frac{\sum_i |x_i|}{\left|\sum_i x_i\right|}}_{\text{求和问题的条件数}}$$

括号里这一项就是求和问题的**条件数**（condition number）〔讲义 Lec 3a 明确称其为 "what we will eventually call the condition number"〕。若所有 $x_i \ge 0$，它等于 1，误差界退化为 $n\epsilon$；若有正有负且和接近 0，它可以任意大——**此时问题本身病态，任何算法都救不了**。

**平均情形**：若各 $\epsilon_k$ 符号随机，$\delta_i$ 是 $n-i+1$ 个随机变量之和，随机游走给出 $\sqrt{n-i+1}\,\epsilon$，故均方根误差 $\sim \sqrt{n}\,\epsilon \sum |x_i|$〔讲义 Lec 3a〕。

**向后稳定性**〔讲义 Lec 3b〕：存在扰动后的输入 $\tilde{x}$ 使 $\tilde{f}(x) = f(\tilde{x})$ 且 $\|\tilde{x} - x\|_1 = \|\tilde{x}\|_1 O(\epsilon)$。构造极简：

$$\tilde{x}_i = \tilde{s}_{i-1} \oplus x_i - \tilde{s}_{i-1} = (\tilde{s}_{i-1} + x_i)(1+\epsilon_i) - \tilde{s}_{i-1}$$

于是 $\tilde{s}_i = \sum_{k \le i} \tilde{x}_k$ 归纳成立。再用 $|\tilde{x}_i - x_i| \le (|x_i| + \|\tilde{x}\|_1)|\epsilon_i|$ 求和、并用 $\|x\| \le \|\tilde{x}\| + \|x - \tilde{x}\|$ 解出

$$\|\tilde{x} - x\|_1 = \frac{\|\tilde{x}\|_1 O(\epsilon)}{1 - O(\epsilon)} = \|\tilde{x}\|_1 O(\epsilon)$$

**这一步的教学价值**：讲义强调"稳定性可以在任意范数下证明，一个范数下稳定则所有范数下稳定"，以及"用 $\|x\|$ 还是 $\|\tilde{x}\|$ 做分母无所谓"。这正是第 4 章范数等价性的直接用途。

## 3.4 更好的算法与代价

| 算法 | 最坏相对误差 | 平均 | 额外内存 | 每元素代价 |
|---|---|---|---|---|
| 朴素顺序求和 | $n\epsilon$ | $\sqrt{n}\epsilon$ | $O(1)$ | 1 次加法 |
| 成对求和（pairwise，Julia `sum` 默认） | $O(\log n)\epsilon$ | $O(\sqrt{\log n})\epsilon$ | $O(\log n)$ 栈 | ≈1 次加法 |
| Kahan 补偿求和 | $2\epsilon$（与 $n$ 无关） | — | 2 个寄存器 | 4 次加法 |
| Neumaier 改进版 | $2\epsilon$ | — | 2 个寄存器 | 4 次加法 + 1 次分支 |

成对求和的 $O(\log n)$ 界来自〔讲义 Lec 3a〕对 Higham (1993) 的引用；Kahan 的 $2\epsilon$ 是补偿求和的经典结论〔教材/常识〕，下面的实验可以佐证量级。

**实测**（$n = 10^6$，$x_i \sim U[0,1)$，用 80 位 `longdouble` 作参照）〔实验〕：

| 方法 | 绝对误差 |
| --- | --- |
| 顺序累加（Python 循环，双精度） | $1.40 \times 10^{-8}$ |
| `numpy.sum`（成对求和） | $< 10^{-10}$（与参照完全一致到最后一位） |
| Kahan 补偿 | $5.82 \times 10^{-11}$ |

注意 $n\epsilon \sum |x_i| \approx 10^6 \times 1.11 \times 10^{-16} \times 5 \times 10^5 \approx 5.5 \times 10^{-5}$，实际误差比最坏界小 3 个数量级——这就是"最坏界悲观、平均 $\sqrt{n}$"的实证。

## 3.5 实现陷阱

1. **`sum()` 的实现依语言而异**：Python 内置 `sum` 是顺序累加（无补偿），NumPy 是成对求和，Julia 是成对求和。跨语言对比数值结果前先查文档。
2. **并行归约改变求和顺序**：GPU 上一个 block 的 reduction 树是成对求和，跨 block 又是顺序累加，所以"同样的代码在 A100 与 H100 上结果不同"是正常的〔推断〕。
3. **补偿求和不能救抵消**：若 $\sum x_i$ 接近 0，条件数巨大，Kahan 只保证"算的是给定那组数的和的精确值"，不保证这个和有物理意义。`math.fsum`（Shewchuk 精确求和）代价是 $O(n)$ 内存与更慢速度，只在"必须一位不差"时用。

## 3.6 本章要点

- 前向误差 $\lesssim$ 条件数 $\times$ 算法误差因子；求和的条件数是 $\sum|x_i| / |\sum x_i|$；顺序求和**向后稳定**（这是它能被接受的理由），但前向误差随 $n$ 线性增长；
- 想又快又准：用成对求和；想绝对准：用 Kahan/Neumaier 或 `math.fsum`。

---

# 第 4 章 向量范数、矩阵范数与范数等价

> 对应材料：Lec 4《Notes on the equivalence of norms》（S. G. Johnson，2012–09–19）。

## 4.1 问题：换一个范数，结论会不会崩？

误差分析里我们反复写 $\|\cdot\|$，但从不指定是哪一个。这合法吗？在有限维空间里合法，因为**所有范数彼此等价**：存在 $0 < C_1 \le C_2$ 使

$$C_1 \|x\|_b \le \|x\|_a \le C_2 \|x\|_b, \qquad \forall x \in V$$

讲义的证明分四步，是本课程里最"实分析"的一页，值得完整复述〔讲义 Lec 4〕。

## 4.2 证明（四步）

**第 1 步：只需对付 $\|\cdot\|_1$（传递性）。** 取基 $e_1,\dots,e_n$，定义 $\|x\|_1=\sum|\alpha_i|$（$x=\sum\alpha_i e_i$）。若 $\|\cdot\|_a$ 与 $\|\cdot\|_{a'}$ 都与 $\|\cdot\|_1$ 等价，则

$$\frac{C_1'}{C_2}\|x\|_a\le\|x\|_{a'}\le\frac{C_2'}{C_1}\|x\|_a$$

**第 2 步：只需考虑 $\|x\|_1=1$。** 齐次性：除以 $\|x\|_1$ 即可。

**第 3 步：任意范数在 $\|\cdot\|_1$ 下连续。** 由三角不等式

$$\big|\|x\|_a-\|x'\|_a\big|\le\|x-x'\|_a\le\|x-x'\|_1\max_i\|e_i\|_a$$

故取 $\delta=\varepsilon/\max_i\|e_i\|_a$ 即可。

**第 4 步：紧集上的极值。** 令 $C_1=\min_{\|u\|_1=1}\|u\|_a$，$C_2=\max_{\|u\|_1=1}\|u\|_a$。由**极值定理**（连续函数在紧集上取到最大最小值），$C_2\ge C_1>0$（因为 $u\neq0$），证毕。

> 第 4 步是整门课唯一"非代数"的地方：$\{u:\|u\|_1=1\}$ 是闭且有界的，故紧；连续函数在紧集上必取到极值，而不是只取到上确界。18.100A 里的 Bolzano–Weierstrass / 极值定理就是这一步的全部内容。这也是为什么"范数等价"在无穷维空间**不成立**：单位球不再紧。

## 4.3 常用范数与诱导矩阵范数

| 名称 | 定义 | 计算代价 | 备注 |
|---|---|---|---|
| $\|x\|_1$ | $\sum_i\|x_i\|$ | $O(n)$ | "出租车范数" |
| $\|x\|_2$ | $\sqrt{\sum_i\|x_i\|^2}$ | $O(n)$，注意上溢/下溢 | 酉不变，最常用 |
| $\|x\|_\infty$ | $\max_i\|x_i\|$ | $O(n)$ | |
| $\|A\|_1$ | $\max_j\sum_i\|a_{ij}\|$ | $O(mn)$ | 最大**列**和 |
| $\|A\|_\infty$ | $\max_i\sum_j\|a_{ij}\|$ | $O(mn)$ | 最大**行**和 |
| $\|A\|_2$ | $\sigma_{\max}(A)$ | $O(mn^2)$（要 SVD） | 需要 SVD，不是初等量 |
| $\|A\|_F$ | $\sqrt{\sum_{ij}\|a_{ij}\|^2}$ | $O(mn)$ | **不是**诱导范数（$\|I\|_F=\sqrt{n}\neq1$） |

诱导范数的定义是 $\|A\|=\max_{x\neq0}\|Ax\|/\|x\|=\max_{\|x\|=1}\|Ax\|$——同样是一个"紧集上取极值"的构造，和第 4 步同源。

## 4.4 条件数为什么与范数无关（量级上）

由等价性，$\kappa_1, \kappa_2, \kappa_\infty$ 之间只差一个与 $n$ 有关的常数，所以"条件数大不大"这个判断是**定性稳健**的；但报数时必须写清是哪个范数，因为常数可能是 $n$ 或 $\sqrt{n}$。常用的两个恒等式〔教材〕：

$$\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}, \qquad \kappa_2(A^{\mathsf{T}}A) = \kappa_2(A)^2$$

第二个式子就是第 11 章"别用正规方程"的全部理由。

## 4.5 实现陷阱

1. `np.linalg.norm(A, 2)` 对矩阵算的是**谱范数**（要 SVD，$O(n^3)$），对大矩阵非常慢；只要 Frobenius 时用 `norm(A,'fro')`。
2. $\|x\|_2$ 的朴素写法 `sqrt(sum(x**2))` 在 $x_i \approx 10^{200}$ 时上溢、$10^{-200}$ 时下溢为 0；应用 `np.linalg.norm`（内部缩放）或 BLAS `nrm2`。
3. 用 $\|A\|_\infty$ 估条件数时别忘了 $\|A^{-1}\|_\infty$ 需要**显式求逆或解 $n$ 个方程组**，代价是 $O(n^3)$；`np.linalg.cond(A, np.inf)` 走的就是 LU 分解后估逆范数。
4. 判断"矩阵是否奇异"不要看 $\det A$：$\det$ 随规模指数缩放，$10^{-20}$ 的行列式可能对应完全良态的矩阵（如 $0.1I_{20}$）。

## 4.6 本章要点

- 有限维空间所有范数等价，证明靠紧性 + 极值定理；诱导范数由"单位球上的最大值"定义，故 $\|A\|_2 = \sigma_{\max}$；
- 误差界里的常数依赖范数选择，但"是否病态"的定性结论不依赖。

---

# 第 5 章 条件数：问题本身的敏感性

> 对应材料：Lec 5（calendar 主题 "Condition numbers"）。该讲在 OCW 上**没有**公开 PDF，本章据 Trefethen & Bau 的 "Conditioning and Condition Numbers" 一讲与标准结论整理，标注为 〔教材〕；数值例子为 〔实验〕。

## 5.1 问题：错的是算法还是问题?

同一个算法在两组数据上一个给 15 位正确、一个给 3 位正确，是算法不稳定吗？不一定。**条件数衡量问题，稳定性衡量算法**，二者相乘才是误差。Trefethen & Bau 的经典图示把这件事讲成一句话：

$$\text{（前向误差）} \lesssim \underbrace{\kappa(A)}_{\text{问题的条件数}} \times \underbrace{O(\epsilon_{\text{machine}})}_{\text{算法的向后稳定性}}$$

## 5.2 定义与基本不等式

对可逆 $A$ 与扰动 $A+\delta A$、$b+\delta b$〔教材〕：

$$\kappa(A) = \|A\|\,\|A^{-1}\| \;\geq\; 1, \qquad \kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

相对误差界（忽略高阶项）：

$$\frac{\|\delta x\|}{\|x\|} \;\leq\; \frac{\kappa(A)}{1-\kappa(A)\|\delta A\|/\|A\|}\left(\frac{\|\delta A\|}{\|A\|}+\frac{\|\delta b\|}{\|b\|}\right)$$

**读法**：右端括号里是"输入被扰动多少"，前面乘的 $\kappa$ 是"放大器"。当 $\kappa\epsilon \gtrsim 1$ 时，界失效（$1-\cdot$ 变负），意思是**这个问题在双精度下已经无法求解**。

**经验法则**：双精度有 $\approx 16$ 位十进制有效数字，解一个条件数 $\kappa$ 的线性系统，大约损失 $\log_{10}\kappa$ 位，剩 $16-\log_{10}\kappa$ 位可信。

## 5.3 例子：Hilbert 矩阵有多坏

$H_{ij}=1/(i+j-1)$ 是教科书级的病态矩阵。实测〔实验，NumPy 2.3.5〕：

| $n$ | $\kappa_2(H_n)$ | $\log_{10}\kappa$ | 双精度下可信位数（估） |
|---|---|---|---|
| 6 | $1.50\times10^{7}$ | 7.2 | ≈9 |
| 8 | $1.53\times10^{10}$ | 10.2 | ≈6 |
| 10 | $1.60\times10^{13}$ | 13.2 | ≈3 |
| 12 | $1.68\times10^{16}$ | 16.2 | ≈0（已不可解） |

$n=12$ 时 $\kappa > \epsilon_{\text{machine}}^{-1}$，意味着**连"这个矩阵是否可逆"都无法在双精度下判断**。这不是 LU 的锅：换任何算法都一样。

## 5.4 条件数不是"行列式小"

常见误解：$\det A$ 小就病态。反例：$A = 10^{-10} I_{100}$，$\det A = 10^{-1000}$，但 $\kappa_2 = 1$，完美良态。行列式是**体积**，条件数是**形状**（最长/最短拉伸之比）。

## 5.5 复杂度

- $\kappa_1, \kappa_\infty$：一次 LU（$\frac{2}{3}n^3$）+ 估 $\|A^{-1}\|$（Hager/Higham 算法，几次三角求解，$O(n^2)$）；
- $\kappa_2$：需要 $\sigma_{\max}, \sigma_{\min}$，通常 $O(n^3)$ 但常数比 LU 大；
- 大规模稀疏矩阵**算不出** $\kappa$（$A^{-1}$ 稠密），只能用迭代法估（如 `scipy.sparse.linalg` 里的估计器或 Lanczos 估极值）。

## 5.6 实现陷阱

1. `np.linalg.cond(A)` 默认 2–范数，内部走 SVD，$n = 10^4$ 时可能比 `solve` 慢两个数量级；调试时用 `cond(A, inf)` 或 `cond(A, 1)` 更快。
2. 别用 `1/det(A)` 当条件数（理由见 5.4）。
3. 条件数是**范数相关**的：$\kappa_\infty$ 与 $\kappa_2$ 可差 $\sqrt{n}$ 倍，报数要带范数。
4. 特征值问题的条件数与 $\kappa(A)$ 无关：正规矩阵的特征值问题条件数为 1，非正规矩阵可以无穷（见第 13 章）。

---

# 第 6 章 Newton 法与求根：收敛阶与停机准则

> 对应材料：Lec 1《Square roots via Newton's method》（S. G. Johnson，2015–02–04）。

## 6.1 问题：$\sqrt{a}$ 怎么算

这是全课的"最小可复现例子"，它同时展示三件事：算法是固有近似的、收敛速度可以定量、浮点精度是最终瓶颈。迭代式（ Babylonian / Newton，可追溯到公元前 1000 年左右的巴比伦，讲义脚注引 Boyer《A History of Mathematics》第 3 章与 YBC 7289 泥板）：

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$$

## 6.2 算法与收敛证明

**单调有界证明**（讲义引 Rudin《Principles of Mathematical Analysis》的路线）〔讲义 Lec 1〕：

1. 若 $x_n > \sqrt{a}$，则 $\sqrt{a} < x_{n+1} < x_n$：因为 $x_{n+1} - x_n = \frac{a - x_n^2}{2x_n} < 0$，且 $x_{n+1}^2 - a = \frac{(x_n^2 - a)^2}{4x_n^2} > 0$；
2. 单调下降且有下界 ⇒ 收敛（Rudin 定理 3.14）；若 $x_1 < \sqrt{a}$，则 $x_2 > \sqrt{a}$，$n \geq 2$ 后同样单调；
3. 极限满足 $x = \frac{1}{2}(x + a/x) \Rightarrow x^2 = a$。

**但单调收敛证明不告诉你有"多快"**——这正是数值分析与实分析的分工。

## 6.3 收敛速率：二次收敛的定量形式

令 $x_n = x(1 + \delta_n)$，$x = \sqrt{a}$，代入并把 $(1 + \delta_n)^{-1}$ 泰勒展开〔讲义 Lec 1〕：

$$1 + \delta_{n+1} = \frac{1}{2}\left(1 + \delta_n + \frac{1}{1 + \delta_n}\right) = 1 + \frac{\delta_n^2}{2} + O(\delta_n^3)$$

即

$$\delta_{n+1} = \frac{\delta_n^2}{2} + O(\delta_n^3)$$

误差**平方**（并折半），等价于**有效数字位数每步翻倍**。讲义给出的 60 位计算序列（$x_1 = 1$）为：

| $n$ | $x_n$（截断） | 正确位数 |
|---|---|---|
| 1 | 1 | 0 |
| 2 | 1.5 | 0 |
| 3 | 1.41666… | 2 |
| 4 | 1.414215686… | 5 |
| 5 | 1.41421356237… | 10 |
| 6 | 1.4142135623730950488… | 20 |
| 7 | 1.41421356237309504880168872420969807856967187537 72… | 40+ |

**七步到 60 位**——这就是二次收敛的威力。推广到一般 $f$：若 $f(x^*) = 0, f'(x^*) \neq 0$，则

$$e_{n+1} = \frac{f''(x^*)}{2f'(x^*)} e_n^2 + O(e_n^3)$$

## 6.4 复杂度

每步 1 次除法 + 1 次加法（$\sqrt{a}$ 情形），$k$ 步得 $2^k$ 位精度 ⇒ 得 $p$ 位需 $O(\log p)$ 步。这是"迭代法优于 $O(p)$ 步逐位算法"的典型。

## 6.5 实现陷阱

1. $x_1 = 0$ **直接除零**；$a = 0$ 时公式退化。生产代码（如 `sqrt` 指令）用查表 + 一到两次 Newton 修正，而不是纯迭代。
2. **多重根退化为线性收敛**：$f(x) = (x-1)^2$ 时 $e_{n+1} \approx e_n/2$，需要改用 $f/f'$ 的修正 Newton 或知道重数。
3. **停机准则不能只看** $|x_{n+1} - x_n| < \text{tol}$：当 $|f'|$ 很小时步长小而误差大；稳妥做法是同时检查 $|f(x_n)|$ 与相对步长，且设最大迭代数。
4. **浮点极限**：当 $x_n$ 与 $\sqrt{a}$ 只差 1 ulp 时，$a/x_n$ 的舍入会让迭代在两个相邻浮点数之间**振荡**，永不满足严格 `<`。所以实现里用 `if x_next == x: break`。
5. **别对** $f$ **的导数做数值差分再喂给 Newton**：差分的相对误差是 $\sqrt{\epsilon} \approx 10^{-8}$，会把二次收敛降级成"最多 8 位"。

---

# 第 7 章 QR 分解 I：投影、Gram–Schmidt 与其失效

> 对应材料：Lec 9（主题 "Solving the normal equations by QR and Gram–Schmidt"）与随附 reading《Gram–Schmidt Orthogonalization》；OCW 未提供 Lec 9 主体 PDF，本章以 reading + Trefethen & Bau 的 "QR Factorization"/"Gram–Schmidt" 两讲为准，实验为笔者实跑。

## 7.1 问题：把一组向量变成标准正交基

给定 $A \in \mathbb{R}^{m\times n}$（$m \ge n$，列满秩），求 $A = QR$，$Q^{\mathsf{T}}Q = I_n$，$R$ 上三角。几何上这是**逐列做正交投影并减掉**；代数上这是把 $A$ 右乘一串上三角矩阵〔讲义 Lec 10 第 2 页〕：

$$A\,R_1 R_2 \cdots R_n = \hat{Q} \;\Longrightarrow\; A = \hat{Q}\,\underbrace{R_1^{-1}\cdots R_n^{-1}}_{R}$$

讲义把 Gram–Schmidt 称为"triangular orthogonalization"（用三角矩阵右乘使列正交），把 Householder 称为"orthogonal triangularization"（用正交矩阵左乘使列三角化）——这两个名字值得记住，它们概括了两类算法的全部差别。

## 7.2 算法：CGS 与 MGS

```
# 经典 Gram–Schmidt (CGS)
for j = 1..n:
    v = a_j
    for i = 1..j-1:  r_ij = q_i^T a_j          # 用原始 a_j 做所有投影
    v = a_j - sum_i r_ij q_i
    r_jj = ||v||;  q_j = v / r_jj

# 修正 Gram–Schmidt (MGS)：唯一的改动是"边算边减"
for j = 1..n:
    v = a_j
    for i = 1..j-1:  r_ij = q_i^T v            # 用已更新的 v
                     v = v - r_ij q_i
    r_jj = ||v||;  q_j = v / r_jj
```

两者在**精确算术下产生完全相同的 $Q, R$**，在浮点下完全不同。

讲义给了一个 4×4 的判例，是"正交性丢失"最干净的证明〔讲义 Lec 9 reading 第 6–7 页〕。取 $a_1 = (1, \epsilon, 0, 0)^{\mathsf{T}}$、$a_2 = (1, 0, \epsilon, 0)^{\mathsf{T}}$、$a_3 = (1, 0, 0, \epsilon)^{\mathsf{T}}$，并近似 $1 + \epsilon^2 \approx 1$：

| 量 | CGS | MGS |
|---|---|---|
| $q_1$ | $(1, \epsilon, 0, 0)^{\mathsf{T}}$ | 同左 |
| $r_{12}, v_2, q_2$ | $r_{12} = q_1^{\mathsf{T}} a_2 = 1$，$v_2 = (0, -\epsilon, \epsilon, 0)^{\mathsf{T}}$，<br>$q_2 = \frac{1}{\sqrt{2}}(0, -1, 1, 0)^{\mathsf{T}}$ | 同左 |
| $r_{23}, v_3$ | $r_{13} = 1$，$v_3 = (0, -\epsilon, 0, \epsilon)^{\mathsf{T}}$，<br>$r_{23} = q_2^{\mathsf{T}} a_3 = 0$ | $r_{13} = 1$，$v_3 = (0, -\epsilon, 0, \epsilon)^{\mathsf{T}}$，$r_{23} = q_2^{\mathsf{T}} v_3 = \epsilon/\sqrt{2}$，<br>故 $v_3 \leftarrow (0, -\frac{\epsilon}{2}, -\frac{\epsilon}{2}, \epsilon)^{\mathsf{T}}$ |
| $q_3$ | $\frac{1}{\sqrt{2}}(0, -1, 0, 1)^{\mathsf{T}}$ | $\frac{1}{\sqrt{6}}(0, -1, -1, 2)^{\mathsf{T}}$ |
| 检验 $q_2^{\mathsf{T}} q_3$ | $\frac{1}{2}$ ❌ | $0$ ✅ |

**读法**：CGS 的正交性误差是 $\frac{1}{2}$ —— **与 $\epsilon$ 无关的 $O(1)$**，不是 $O(\epsilon)$。根源在 $r_{23}$：CGS 用**未更新**的 $a_3$ 算投影，得 $0$；但 $v_3 = a_3 - r_{13} q_1$ 在减去 $q_1$ 后**新出现**了一个 $\epsilon/\sqrt{2}$ 的 $q_2$ 分量（$q_2^{\mathsf{T}} q_1 = -\epsilon/\sqrt{2} \neq 0$，正是 $1 + \epsilon^2 \approx 1$ 的代价），CGS 从未把它减掉。MGS 用**已更新**的 $v_3$ 算 $r_{23}$，才看得见并减掉它。

## 7.3 误差分析：CGS 丢正交性，MGS 不丢

经典结论〔教材〕：CGS 的正交性损失为 $O(\kappa(A)^2 \epsilon)$，MGS 为 $O(\kappa(A)\epsilon)$；但**两者的 $\|A - QR\|/\|A\|$ 都是 $O(\epsilon)$**（都是向后稳定的）。这个"残差小但 $Q$ 不正交"的组合是最反直觉的一条。

实测〔实验〕：$m = 200, n = 60$，随机奇异值按对数均匀分布构造指定 $\kappa$：

| $\kappa(A)$ | CGS $\|I - Q^{\mathsf{T}}Q\|$ | MGS $\|I - Q^{\mathsf{T}}Q\|$ | CGS 残差 | MGS 残差 |
|---|---|---|---|---|
| $10^4$ | $3.1 \times 10^{-9}$ | $2.4 \times 10^{-12}$ | $2.7 \times 10^{-17}$ | $2.7 \times 10^{-16}$ |
| $10^8$ | $3.07$（完全失效） | $1.4 \times 10^{-8}$ | $1.4 \times 10^{-17}$ | $2.7 \times 10^{-16}$ |
| $10^{12}$ | $22.9$ | $8.5 \times 10^{-5}$ | $1.0 \times 10^{-17}$ | $2.5 \times 10^{-16}$ |

**读法**：$\kappa = 10^8$ 时 CGS 的 $Q$ 连"正交"都算不上（$\|I - Q^{\mathsf{T}}Q\| \approx 3$），但它仍然满足 $A \approx QR$ 到机器精度。这说明**残差小 ≠ 解对**：如果你后续要用 $Q$ 做投影（最小二乘），CGS 会给你完全错误的答案。

## 7.4 复杂度

CGS 与 MGS 均为 $2mn^2$ flop（每个 $r_{ij}$ 一次点积 $2m$，共 $n^2/2$ 个；更新再 $2mn^2/2$）。MGS 的常数与 CGS 相同，**没有任何理由在生产代码里用 CGS**。

## 7.5 实现陷阱

1. **用 MGS 但按列存 $A$（column-major）**：MGS 的内层是 `v -= r*q_i` 的 axpy，对列主序极友好；写成行主序会慢 2–5 倍〔推断〕。
2. **不重正交化**：MGS 在 $\kappa > 10^{10}$ 时也会丢正交性，此时需 **MGS 两遍**（"twice is enough"，经典结论〔教材〕），代价翻倍但正交性恢复到 $O(\epsilon)$。
3. **用 `Q,R = np.linalg.qr(A)` 后自己算 $Q^{\mathsf{T}}Q$ 检查**：LAPACK 的 `geqrf` 用 Householder，不是 MGS，所以你会看到 $O(\epsilon)$ 而非 $O(\kappa\epsilon)$。
4. **把 Gram–Schmidt 用于最小二乘的"正规方程"路线**（$R^{\mathsf{T}}Rx = A^{\mathsf{T}}b$）：等于把 $\kappa$ 平方，务必直接解 $Rx = Q^{\mathsf{T}}b$（第 11 章）。
5. **$r_{jj} \approx 0$ 的判断**：列几乎线性相关时 $r_{jj}$ 可能是舍入噪声，除以它会放大误差；应设阈值（如 $r_{jj} \le n\epsilon\|A\|$ 就报秩亏）。

---

# 第 8 章 QR 分解 II：Householder 反射与 Givens 旋转

> 对应材料：Lec 10《Householder Reflectors and Givens Rotations》（Per-Olof Persson 署名幻灯片，2006-09-26）。

## 8.1 问题：要"三角化"，不要"正交化"

Gram–Schmidt 是"右乘三角矩阵把列变正交"，数值上会丢正交性。反过来做：**左乘正交矩阵把列变三角**。每一步用一个反射（reflector）把一列的尾部一次性清零，且**不破坏已经造好的零**〔讲义 Lec 10〕。

## 8.2 算法：Householder 反射

取 $Q_k = \begin{bmatrix} I & 0 \\ 0 & F \end{bmatrix}$，$F = I - 2\dfrac{vv^*}{v^*v}$ 是 Householder 反射（$v$ 为镜面法向）。要让 $Fx = \|x\|e_1$，取

$$v = \mathbf{sign}(x_1)\|x\|_2 e_1 + x$$

**为什么带 $\mathbf{sign}(x_1)$**：若取 $v = \|x\|e_1 - x$，当 $x_1 > 0$ 时 $v$ 的两个大分量相减，发生灾难性抵消，$\|v\|$ 的相对误差被放大；带符号后 $v$ 的两项**同号相加**，数值稳定〔讲义 Lec 10 第 7 页〕。

```
# Householder QR：就地存 R，另存反射向量 v_k
for k = 1 to n:
    x = A[k:m, k]
    v_k = sign(x_1) * ||x||_2 * e_1 + x
    v_k = v_k / ||v_k||_2
    A[k:m, k:n] = A[k:m, k:n] - 2 v_k (v_k^* A[k:m, k:n])
```

应用 $Q$ 或 $Q^*$ 时**不要显式形成 $Q$**〔讲义 Lec 10 第 9 页〕：

```
# 计算 Q^* b = Q_n ... Q_2 Q_1 b
for k = 1 to n:      b[k:m] -= 2 v_k (v_k^* b[k:m])
# 计算 Q x = Q_1 Q_2 ... Q_n x
for k = n downto 1:  x[k:m] -= 2 v_k (v_k^* x[k:m])
```

## 8.3 误差与稳定性

- Householder QR 是**向后稳定**的：存在精确正交的 $\tilde{Q}$ 使 $A + \delta A = \tilde{Q}\tilde{R}$，$\|\delta A\|/\|A\| = O(\epsilon)$〔教材〕；
- 显式形成 $Q$ 时，正交性误差为 $O(n\epsilon)$（因为累乘了 $n$ 个反射），与 $\kappa(A)$ 无关——这是它相对 CGS 的决定性优势；
- 用 Householder QR 解最小二乘，前向误差 $\sim \kappa(A)\epsilon$，而正规方程是 $\kappa(A)^2\epsilon$。

## 8.4 复杂度

讲义给出的 flop 数〔讲义 Lec 10 第 10 页〕：每步 $4(m-k)(n-k)$，求和得

$$\sum_{k=1}^{n} 4(m-k)(n-k) = 2mn^2 - \frac{2n^3}{3} + O(mn)$$

对比 Givens：每个旋转只消一个元素，共约 $mn$ 个旋转，每个 $6m$ 左右的代价，总计

$$3mn^2 - n^3$$

即 **Givens 比 Householder 贵 50%**〔讲义 Lec 10 第 12 页〕。

## 8.5 Givens 什么时候值得用

| 场景 | 选谁 | 理由 |
|---|---|---|
| 稠密 $m \times n$，$m \gg n$ | Householder | flop 少一半，易分块 |
| 稀疏/带状矩阵（如 QR 迭代中的 Hessenberg） | Givens | 每个旋转只碰 2 行，不引入新填充 |
| 需要增量更新（加一行/删一行） | Givens | 只影响局部 |
| 并行/流水线 | Givens | 旋转之间可流水，Householder 是整列操作 |

## 8.6 实现陷阱

1. **`sign(0)` 的坑**：讲义明确提醒 `sign(0)=1`，而 MATLAB 的 `sign(0)==0`〔讲义 Lec 10 第 7 页〕。若直接用 `sign(x1)` 而 $x_1 = 0$，$v = 0$，除以零。正确写法是 `s = 1 if x1 >= 0 else -1`。
2. **用 $\|v\|_2$ 的朴素平方和**：$x$ 的分量跨越 $10^{\pm 150}$ 时会溢出；用缩放版 `hypot` / `nrm2`。
3. **显式形成 $Q$ 再乘**：$O(m^2 n)$ 内存 + $O(m^2 n)$ flop，比隐式应用贵一个量级；LAPACK 的 `orgqr` 只在确实需要 $Q$ 时调用。
4. **忽略 $\|v\|$ 归一化**：很多实现省掉 $v \leftarrow v/\|v\|$，改用 $F = I - \beta v v^{\mathsf{T}}$ 存 $\beta$；自己实现时若混用两种约定，$Q$ 会差一个尺度。

---

# 第 9 章 LU 分解、部分主元与增长因子

> 对应材料：Lec 13（calendar："LU factorization and partial pivoting"），OCW 无该讲 PDF。本章据 Trefethen & Bau 的 "Gaussian Elimination"/"Pivoting"/"Stability of Gaussian Elimination" 三讲与标准结论整理，标注 〔教材〕；数值例子为 〔实验〕。

## 9.1 问题：一次分解，多次求解

解 $Ax = b$。若只有右端一个 $b$，消元的代价是 $\frac{2}{3}n^3$；若有 $k$ 个右端，**先分解再回代**总代价是 $\frac{2}{3}n^3 + 2kn^2$，而逐个消元是 $k \cdot \frac{2}{3}n^3$。这就是"分解"的工程意义：$k \geq 2$ 时就该分解。

## 9.2 算法：带部分主元的高斯消元

```
U = A; L = I; P = I
for k = 1..n-1:
    p = argmax_{i>=k} |U[i,k]|          # 部分主元：只在当前列找
    swap rows k,p of U, L[:, :k], and P
    for i = k+1..n:
        L[i,k] = U[i,k] / U[k,k]
        U[i, k:] -= L[i,k] * U[k, k:]
```

得到 $PA = LU$（$L$ 单位下三角，$U$ 上三角），然后解 $Ly = Pb$、$Ux = y$。

## 9.3 为什么必须选主元：一个 2×2 的反例

取 $A = \begin{bmatrix} \epsilon & 1 \\ 1 & 1 \end{bmatrix}$，$b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$，真解 $x = (1,1)^{\mathsf{T}}$，$\epsilon = 10^{-17}$〔实验〕：

| 策略 | 计算过程 | 结果 |
|---|---|---|
| 不选主元 | $l_{21} = 1/\epsilon = 10^{17}$，$u_{22} = 1 - 10^{17} \approx -10^{17}$，$b_2' = 2 - 10^{17} \approx -10^{17}$ | $x = (0,\ 1)^{\mathsf{T}}$ ❌ |
| 部分主元 | 先交换两行，$l_{21} = \epsilon$，$u_{22} = 1 - \epsilon \approx 1$ | $x = (1,\ 1)^{\mathsf{T}}$ ✅ |

不选主元时 $x_1$ 的**相对误差 100%**，原因正是第 2 章的灾难性抵消：$u_{22}$ 与 $b_2'$ 都由两个 $10^{17}$ 量级的数相减得到，各自的 1 被完全吞掉。

## 9.4 增长因子：主元能选多好?

定义增长因子 $\rho = \dfrac{\max_{i,j}|u_{ij}|}{\max_{i,j}|a_{ij}|}$。部分主元在最坏情况下仍有 $\rho = 2^{n-1}$：取 $A$ 的对角线全 1、下三角全 $-1$、最后一列全 1，则 $u_{nn} = 2^{n-1}$〔教材〕。实测〔实验〕$n = 8$ 时 $\rho = 128 = 2^7$，完全吻合。这意味着**部分主元不是万能的**：理论上 $n = 64$ 时 $\rho \approx 9 \times 10^{18}$，早已超过 $1/\epsilon$。实践中这种矩阵极少出现，故 LAPACK 仍默认部分主元。

## 9.5 误差与稳定性

- 带部分主元的 GE 是**向后稳定**的：$\hat{L}\hat{U} = P(A+\delta A)$，$\|\delta A\| \le c(n)\rho\epsilon\|A\|$〔教材〕；
- 前向误差 $\sim \kappa(A)\rho\epsilon$；
- **主元不改善条件数**：$\kappa(PA) = \kappa(A)$，选主元只是避免算法自身放大误差。

## 9.6 复杂度

| 操作 | flop |
|---|---|
| LU 分解（含主元） | $\frac{2}{3}n^3 + O(n^2)$ |
| 前代 $Ly = Pb$ | $n^2$ |
| 回代 $Ux = y$ | $n^2$ |
| 求逆 $A^{-1}$（解 $n$ 个右端） | $\frac{8}{3}n^3$ |
| 行列式 | $\det A = (-1)^s \prod u_{ii}$，$O(n^3)$ 已有分解时 $O(n)$ |

## 9.7 实现陷阱

1. **自己写 `inv(A) @ b`**：先求逆再乘是 $\frac{8}{3}n^3 + 2n^2$，比 `solve` 贵 4 倍且误差更大。
2. **用 `np.linalg.det(A) == 0` 判奇异**：行列式可能因上溢/下溢变成 `inf` / `0.0`，与是否奇异无关；看 $U$ 的对角元或 `cond`。
3. **对称矩阵也用 LU**：对称正定该用 Cholesky（省一半 flop、更稳）；对称不定该用 $LDL^{\mathsf{T}}$ 或 Bunch–Kaufman。
4. **带状矩阵用稠密 LU**：带宽 $p$ 的矩阵 LU 只需 $O(np^2)$，稠密分解是 $O(n^3)$，差几个数量级。
5. **稀疏矩阵直接调 `scipy.linalg.lu`**：会把稀疏结构炸成稠密，必须用 `scipy.sparse.linalg.splu`（第 20 章）。

---

# 第 10 章 Cholesky 分解与结构矩阵

> 对应材料：Lec 14（calendar："Cholesky factorization and other specialized solvers"），OCW 无 PDF。本章据 Trefethen & Bau 的 "Cholesky Factorization" 一讲与标准结论整理，标注 〔教材〕。

## 10.1 问题：对称正定矩阵的最优分解

$A = A^{\mathsf{T}} \succ 0$ 时，存在唯一的 $A = R^{\mathsf{T}}R$（$R$ 上三角、对角为正），或 $A = LL^{\mathsf{T}}$（$L$ 下三角）。称 Cholesky 分解。它有三个"免费的好处"：

1. **不需要选主元**：正定性保证对角元恒正，$|l_{ik}| \le \sqrt{a_{kk}}$，不会增长；
2. **代价减半**：$\frac{1}{3}n^3$ 而非 $\frac{2}{3}n^3$；
3. **天然给出"平方根"**：条件数关系 $\kappa_2(A) = \kappa_2(R)^2$，所以 $\kappa(R) = \sqrt{\kappa(A)}$。

## 10.2 算法

```
for j = 1..n:
    L[j,j] = sqrt(A[j,j] - sum_{k<j} L[j,k]^2)
    for i = j+1..n:
        L[i,j] = (A[i,j] - sum_{k<j} L[i,k]L[j,k]) / L[j,j]
```

按列（column-oriented，LAPACK `potrf` 的做法）写对缓存友好；按行写（"外积"形式）更好分块。

## 10.3 误差与稳定性

- 无主元 Cholesky 是**向后稳定**的：$\hat{R}^{\mathsf{T}}\hat{R} = A + \delta A$，$\|\delta A\|/\|A\| = O(\epsilon)$〔教材〕；
- 前向误差 $\sim \kappa(A)\epsilon$；
- **判断正定性用 Cholesky，不要用特征值**：`cholesky` 失败即非正定（或数值上非正定），代价 $\frac{1}{3}n^3$；`eigvalsh` 是 $4n^3/3$ 量级且不告诉你"离正定有多远"。更严格的是看最小特征值或 $L$ 的最小对角元。
- **正定性判据的陷阱**：$A$ 对称且对角元全正**不**蕴含正定；$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ 的对角元全正但特征值为 $-1, 3$。

## 10.4 复杂度

| 分解 | flop | 是否需主元 | 适用 |
|---|---|---|---|
| $A = LL^{\mathsf{T}}$ | $\frac{1}{3}n^3$ | 否 | SPD |
| $A = LDL^{\mathsf{T}}$ | $\frac{1}{3}n^3$ | 否（SPD）/ 需 1×1&2×2 块（不定） | 对称，避免开方 |
| $PA = LU$ | $\frac{2}{3}n^3$ | 是 | 一般方阵 |
| 带状 Cholesky（带宽 $p$） | $O(np^2)$ | 否 | 带状 SPD |

## 10.5 实现陷阱

1. **`scipy.linalg.cholesky` 默认返回上三角 $R$**，`numpy.linalg.cholesky` 返回下三角 $L$；两者互为转置，混用会得到 $A = R^{\mathsf{T}}R$ 与 $A = LL^{\mathsf{T}}$ 的混乱。
2. **只给下三角却传了完整矩阵**：`scipy.linalg.cholesky(A, lower=True)` 只读下三角，若上三角是垃圾数据也不报错——务必保证对称。
3. **把 Cholesky 用于半正定（奇异）矩阵**：对角元会变成 0 或负数，需改用带主元的 $LDL^{\mathsf{T}}$ 或截断（pivoted Cholesky）。
4. **用 Cholesky 解最小二乘的正规方程**：$A^{\mathsf{T}}A$ 的条件数是 $\kappa(A)^2$，且形成 $A^{\mathsf{T}}A$ 本身就会丢信息（第 11 章）。
5. **带状矩阵当稠密存**：$n = 10^5$、带宽 3 的三对角矩阵，稠密存储要 80 GB，带状只要几 MB。

---

# 第 11 章 线性最小二乘：正规方程 vs QR

> 对应材料：Lec 8（"Linear regression and the generalized SVD"）与 Lec 9（"Solving the normal equations by QR and Gram–Schmidt"），OCW 仅有 Lec 9 的 reading。本章据 Trefethen & Bau 的 "Least Squares" 一讲整理，实验为笔者实跑。

## 11.1 问题

给定 $A \in \mathbb{R}^{m\times n}$（$m > n$，列满秩）、$b \in \mathbb{R}^m$，求

$$\hat{x} = \arg\min_x \|Ax - b\|_2$$

正规方程（normal equations）$A^{\mathsf{T}}A\hat{x} = A^{\mathsf{T}}b$ 是"教科书答案"，也是**数值上最不该用的答案**。

## 11.2 两条路线

**路线 A：正规方程。** 形成 $A^{\mathsf{T}}A$（$mn^2$ flop），Cholesky 分解（$\frac{1}{3}n^3$），解。总代价约 $mn^2 + \frac{1}{3}n^3$，但

$$\kappa_2(A^{\mathsf{T}}A) = \kappa_2(A)^2$$

**路线 B：QR。** $A = QR$，则 $\|Ax - b\|^2 = \|Rx - Q^{\mathsf{T}}b\|^2 + \|(I - QQ^{\mathsf{T}})b\|^2$，故解 $R\hat{x} = (Q^{\mathsf{T}}b)_{1:n}$。代价 $2mn^2 - \frac{2}{3}n^3$，误差 $\sim \kappa(A)\epsilon$。

## 11.3 实测：Hilbert 矩阵上的最小二乘

取 $A = H_n$（$n \times n$ Hilbert），$x_{\text{true}} = \mathbf{1}$，$b = Ax_{\text{true}}$，比较两种解法的相对误差〔实验〕：

| $n$ | $\kappa(A)$ | $\kappa(A^{\mathsf{T}}A)$ | 正规方程相对误差 | QR 相对误差 |
|---|---|---|---|---|
| 8 | $1.5 \times 10^{10}$ | $1.0 \times 10^{18}$ | $2.76$（276%！） | $6.5 \times 10^{-8}$ |
| 10 | $1.6 \times 10^{13}$ | $2.4 \times 10^{18}$ | $5.34$ | $3.6 \times 10^{-4}$ |
| 12 | $1.7 \times 10^{16}$ | $3.8 \times 10^{17}$ | $27.8$ | $9.0 \times 10^{-2}$ |

**结论**：$\kappa(A)$ 一旦超过 $10^8$，正规方程就基本报废；QR 还能撑到 $\kappa \approx 1/\epsilon$。这不是实现问题，是 $\kappa^2$ 的数学必然。

## 11.4 更稳的做法与代价阶梯

| 方法 | flop | 误差量级 | 秩亏时 |
|---|---|---|---|
| 正规方程 + Cholesky | $mn^2 + \frac{1}{3}n^3$ | $\kappa^2\epsilon$ | 崩 |
| Householder QR | $2mn^2 - \frac{2}{3}n^3$ | $\kappa\epsilon$ | 崩（$R$ 对角为 0） |
| 列主元 QR（QR with column pivoting） | $2mn^2$ | $\kappa\epsilon$，能识别数值秩 | 可给基本解 |
| SVD / 截断 SVD | $O(mn^2)$，常数约 2× QR | $\epsilon$（对截断子空间） | 稳，给最小范数解 |

## 11.5 实现陷阱

1. **`np.linalg.lstsq` 默认用 SVD（`rcond=None` 时按机器精度截断）**，比 QR 慢但最稳；要快可传 `lapack_driver='gelsy'`（列主元 QR）。
2. **自己写 `solve(A.T@A, A.T@b)`**：这就是路线 A，除非 $\kappa(A) < 10^4$ 否则别用。
3. **忘记加截距列**：线性回归的"设计矩阵"要含一列 1，否则解的是过原点模型。
4. **用 `pinv` 处理秩亏**：`pinv` 给最小范数解，但阈值 `rcond` 的选择会显著改变结果；需明确"多小的奇异值算零"。
5. **加权最小二乘**：$\min \|W^{1/2}(Ax - b)\|$，应把 $W^{1/2}$ 乘进 $A, b$ 再做 QR，而不是形成 $A^{\mathsf{T}}WA$。

---

# 第 12 章 SVD：几何、条件数与低秩逼近

> 对应材料：Lec 7（"The SVD, its applications, and condition numbers"），OCW 无 PDF。本章据 Trefethen & Bau 的 "The Singular Value Decomposition"/"More on the SVD" 两讲整理，标注〔教材〕；截断实验为〔实验〕。

## 12.1 问题：任何矩阵都是"旋转—拉伸—旋转"

对 $A \in \mathbb{C}^{m\times n}$，存在酉矩阵 $U \in \mathbb{C}^{m\times m}$、$V \in \mathbb{C}^{n\times n}$ 与对角 $\Sigma \in \mathbb{R}^{m\times n}$（$\sigma_1 \geq \cdots \geq \sigma_{\min} \geq 0$）使

$$A = U\Sigma V^*, \qquad A = \sum_{i=1}^{r} \sigma_i u_i v_i^* \ (r = \operatorname{rank} A)$$

几何意义：单位球被映成椭球，$\sigma_i$ 是半轴长。这解释了为什么 $\|A\|_2 = \sigma_1$、$\|A\|_F = \sqrt{\sum \sigma_i^2}$、$\kappa_2 = \sigma_1/\sigma_n$。

## 12.2 与特征分解的关系（以及为什么不能直接对 $A$ 求特征值）

- $A^*A = V\Sigma^{\mathsf{T}}\Sigma V^*$，$AA^* = U\Sigma\Sigma^{\mathsf{T}}U^*$；
- **数值上不要**通过 $A^{\mathsf{T}}A$ 的特征分解算 SVD：$\kappa$ 被平方（同第 11 章的病）；
- $A$ 的奇异值 = $\sqrt{A^{\mathsf{T}}A}$ 的特征值；但 $A$ 的特征值可以是复数、可以对扰动极其敏感（第 13 章），二者无稳定关系。

## 12.3 Eckart—Young：低秩逼近的最优性

$$\min_{\operatorname{rank}(B)\leq k} \|A-B\|_2 = \sigma_{k+1}, \qquad \min_{\operatorname{rank}(B)\leq k} \|A-B\|_F = \sqrt{\sum_{i>k}\sigma_i^2}$$

**实测**〔实验〕：$50\times 30$ 高斯随机矩阵，检查截断误差是否等于"尾巴"：

| $k$ | $\|A-A_k\|_F/\|A\|_F$ | $\sqrt{\sum_{i>k}\sigma_i^2}/\|A\|_F$ |
|---|---|---|
| 1 | 0.9493 | 0.9493 |
| 10 | 0.5727 | 0.5727 |

两者**逐位相同**，验证了 Eckart—Young。注意这个矩阵的 $\sigma_1/\sigma_{30} = 5.85$，谱很平，所以低秩逼近效果差——**"能不能压缩"由奇异值衰减速度决定，不由 $k$ 决定**。

## 12.4 复杂度

- 稠密 $m \times n$（$m \geq n$）：Golub–Kahan 先双对角化（$4mn^2 - \frac{4}{3}n^3$ 量级），再对双对角阵做隐式 QR，总 $O(mn^2)$；常数约为 Householder QR 的 2 倍〔教材/量级估计〕；
- 只要前 $k$ 个奇异三元组：随机化 SVD 或 Lanczos 双对角化，$O(mnk)$；
- 稀疏大矩阵：`scipy.sparse.linalg.svds`（ARPACK/Lanczos），只做 matvec。

## 12.5 实现陷阱

1. `np.linalg.svd(A)` **默认** `full_matrices=True`，$m \gg n$ 时会返回 $m \times m$ 的 $U$，内存爆炸；用 `full_matrices=False`。
2. **用** `A.T@A` **的特征值开方当奇异值**：$\sigma$ 的相对误差被放大成 $\kappa^2\epsilon$，小奇异值甚至变成虚数（负特征值开方）。
3. `svd` **的符号不唯一**：$u_i, v_i$ 同时变号仍是同一分解；比较两次运行的 $U$ 必须按 $|u_i^{\mathsf{T}}\tilde{u}_i|$ 比。
4. **截断阈值拍脑袋**：常见做法是"保留 99% 能量"（$\sum_{i\leq k}\sigma_i^2 \geq 0.99\sum\sigma_i^2$）或按 $\sigma_k > \tau\sigma_1$；不同阈值给出完全不同的秩，必须写进实验记录。
5. **复数矩阵用** `A.T` **而非** `A.conj().T`：SVD 里的 $V^*$ 是共轭转置，用转置会得到错误的奇异向量。

---

# 第 13 章 特征值问题 I：Schur 形式与 Hessenberg 归约

> 对应材料：Lec 15《Hessenberg/Tridiagonal Reduction》（Per-Olof Persson 署名幻灯片，幻灯片页眉作 "Lecture 14"）与 Lec 14（calendar 覆盖 "Eigenproblems and Schur factorizations"）。

## 13.1 问题：为什么不能像解线性系统那样"一次做完"

线性系统有直接法（LU，有限步精确结束）；特征值问题没有。Abel–Galois 定理说五次以上多项式无根式解，而特征多项式 $\det(A - \lambda I)$ 可以是任意多项式——所以**任何特征值算法都必须是迭代的**。讲义在讲 Hessenberg 归约时用一句话点破了"直接造对角形"的失败原因〔讲义 Lec 15 第 2 页〕：右乘会破坏左乘造出的零，且"我们早就知道这不会成功，因为 Abel 定理"。

## 13.2 Schur 分解：迭代的目标形态

对任意 $A \in \mathbb{C}^{n\times n}$，存在酉 $Q$ 使

$$A = QTQ^*, \qquad T \text{ 上三角（对角为特征值）}$$

$A$ 实对称时 $T$ 可取实对角（谱定理）。所以特征值算法的目标不是"对角化"，而是"**酉相似到上三角**"，实矩阵则是"实 Schur 形"（对角线上允许 $2\times 2$ 块，对应复共轭特征值对）。

## 13.3 特征值问题的条件数

单个**单重**特征值 $\lambda$ 的条件数〔教材〕：

$$\kappa(\lambda) = \frac{1}{|y^* x|}$$

其中 $x$ 是右特征向量、$y$ 是左特征向量（$y^*A = \lambda y^*$）。正规矩阵 $y \parallel x$，$\kappa = 1$；非正规矩阵 $x, y$ 近正交，$\kappa$ 可以任意大——**即便 $\kappa(A) = 1$**。这与第 5 章的矩阵条件数是完全不同的量，务必分开记。

**退化（defective）情形更糟**：$J = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ 只有一个特征向量，摄动 $\|E\| = 10^{-16}$ 可使特征值移动 $\sqrt{\epsilon} = 10^{-8}$〔实验：`eigvals(J + 1e-16*ones)` 给出 $1 \pm 10^{-8}$〕——**误差是摄动的平方根**，从 16 位掉到 8 位。

## 13.4 算法：Householder 归约到 Hessenberg / 三对角

思路：用相似变换 $Q^*AQ$ 逐列清零，但**左右同时乘**，于是只清到"次对角线以下"：

```
for k = 1 to m-2:
    x = A[k+1:m, k]
    v_k = sign(x_1)||x||_2 e_1 + x;  v_k /= ||v_k||_2
    A[k+1:m, k:m]   -= 2 v_k (v_k^* A[k+1:m, k:m])     # 左乘
    A[1:m, k+1:m]   -= 2 (A[1:m, k+1:m] v_k) v_k^*     # 右乘（对称地）
```

$m-2$ 步后得 $H = Q^*AQ$ 上 Hessenberg（$h_{ij} = 0$ 当 $i > j+1$）。若 $A$ 是 Hermite 的，对称性自动把上三角部分也清零，得到**三对角** $T$。

## 13.5 误差与稳定性

- Householder Hessenberg 归约是**向后稳定**的：$\tilde{Q}\tilde{H}\tilde{Q}^* = A + \delta A$，$\|\delta A\|/\|A\| = O(\epsilon)$，$\tilde{Q}$ 精确酉〔讲义 Lec 15 第 6 页〕；
- 归约**不改变特征值**（相似变换），但**会改变特征值问题的条件数**吗？不会——特征值是相似不变量，$\kappa(\lambda)$ 也是；
- 归约后每步 QR 迭代的代价从 $O(m^3)$ 降到 $O(m^2)$，这是归约的全部意义。

## 13.6 复杂度

讲义给出〔讲义 Lec 15 第 5 页〕：

| 归约 | flop | 说明 |
|---|---|---|
| 一般矩阵 → Hessenberg | $\sum_k \left[4(m-k)^2 + 4m(m-k)\right] = \frac{10m^3}{3}$ | 左乘 + 右乘各一遍 |
| Hermite → 三对角 | $\frac{4m^3}{3}$ | 对称性省一半，且右乘可复用左乘信息 |

注意这个常数比 Householder QR 的 $\frac{2}{3}m^3$ 大 5 倍——**做特征值比做 QR 分解贵得多**，这是"特征值算法是迭代的"之外的第二重代价。

## 13.7 实现陷阱

1. **用 `np.linalg.eig` 对对称矩阵求特征值**：它走通用 QR 算法，结果可能有微小虚部；对称/Hermite 必须用 `eigvalsh` / `eigh`（走三对角 + 隐式 QR，快 2–4 倍且保证实特征值）。
2. **显式形成 $Q$ 再算 $Q^*AQ$**：$O(m^3)$ 且多一次舍入；应该边归约边累积，或用 LAPACK 的 `gehrd` + `orghr`。
3. **把 Hessenberg 归约当成"求特征值"**：它只做归约，特征值要靠后续 QR 迭代（第 15 章）。
4. **对稀疏矩阵做 Hessenberg 归约**：会彻底破坏稀疏性（填充到稠密），大稀疏矩阵必须走 Arnoldi/Lanczos（第 16 章）。
5. **忽视"实 Schur 形有 2×2 块"**：从 `scipy.linalg.schur` 拿到的 $T$ 不是对角阵，直接取 `diag(T)` 会漏掉复特征值。

---

# 第 14 章 特征值问题 II：幂法、反迭代与 Rayleigh 商迭代

> 对应材料：Lec 16 的 reading《The QR Algorithm I》（Per-Olof Persson 署名，2006-10-31），第 3–8 页讲 Rayleigh 商、幂法、反迭代、RQI。

## 14.1 问题：只要一个特征值/特征向量

很多应用（PageRank、稳定性分析、谱聚类）只要**极端**特征值。此时不该做 $O(n^3)$ 的完整分解，而该做"反复乘矩阵"。

## 14.2 Rayleigh 商：把向量变成特征值的估计

$$r(x) = \frac{x^{\mathsf{T}} A x}{x^{\mathsf{T}} x}$$

三条性质〔讲义 reading I 第 3 页〕：

1. $x$ 是特征向量 $\iff$ $r(x) = \lambda$ 且 $\nabla r(x) = 0$（$x \neq 0$）；
2. 对任意 $x$，$r(x)$ 是使 $\|Ax - \alpha x\|_2$ 最小的 $\alpha$；
3. **二次精度**：$r(x) - r(q_J) = O(\|x - q_J\|^2)$——即使特征向量只有 8 位准，特征值也能有 16 位准。这条性质是 Rayleigh 商迭代三次收敛的根源。

## 14.3 幂法及其收敛率

```
v^(0) = 任意单位向量
for k = 1,2,...:
    w = A v^(k-1)
    v^(k) = w / ||w||
    λ^(k) = (v^(k))^T A v^(k)
```

把 $v^{(0)} = \sum_j a_j q_j$ 代入〔讲义 reading I 第 5 页〕：

$$v^{(k)} = c_k \lambda_1^k \left( a_1 q_1 + a_2 \left(\frac{\lambda_2}{\lambda_1}\right)^k q_2 + \cdots \right)$$

故若 $|\lambda_1| > |\lambda_2| \geq \cdots$ 且 $q_1^{\mathsf{T}} v^{(0)} \neq 0$：

$$\|v^{(k)} - (\pm q_1)\| = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right), \qquad |\lambda^{(k)} - \lambda_1| = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^{2k}\right)$$

**注意指数是 $2k$**：特征值误差按比值的平方收敛（因为 Rayleigh 商二次精度），这常被忽略。

**实测**〔实验〕：$n = 100$ 对称矩阵，$\lambda_1 = 1$、$\lambda_2 = 0.99495$（比值 0.994949），从随机初值出发：

| 迭代 $k$ | $\lambda^{(k)}$ | $1 - \lambda^{(k)}$ |
|---|---|---|
| 10 | 0.962824 | $3.7 \times 10^{-2}$ |
| 50 | 0.990536 | $9.5 \times 10^{-3}$ |
| 100 | 0.996188 | $3.8 \times 10^{-3}$ |

误差从 $3.7 \times 10^{-2}$ 到 $3.8 \times 10^{-3}$，100 步只降一个数量级——**比值 0.995 时幂法几乎不动**。实践判据：要 $d$ 位精度需 $k \approx d \log 10 / \log(|\lambda_1/\lambda_2|)$ 步。

## 14.4 反迭代与 Rayleigh 商迭代

**反迭代**：对 $(A-\mu I)w=v$ 做幂法，等价于对 $(A-\mu I)^{-1}$ 做幂法，特征值变为 $(\lambda_j-\mu)^{-1}$。若 $\mu$ 靠近 $\lambda_J$，则 $|\mu-\lambda_J|/|\mu-\lambda_K|$ 很小，收敛极快〔讲义 reading I 第 6 页〕：

$$\|v^{(k)}-(\pm q_J)\|=O\left(\left|\frac{\mu-\lambda_J}{\mu-\lambda_K}\right|^k\right)$$

**Rayleigh 商迭代（RQI）**：每步把 $\mu$ 更新为当前 Rayleigh 商〔讲义 reading I 第 7–8 页〕：

```
v^(0) 单位向量; λ^(0) = r(v^(0))
for k = 1,2,...:
    solve (A − λ^(k−1) I) w = v^(k−1)
    v^(k) = w/||w||
    λ^(k) = r(v^(k))
```

收敛是**三次**的：$\|v^{(k+1)}-(\pm q_J)\|=O(\|v^{(k)}-(\pm q_J)\|^3)$，$|\lambda^{(k+1)}-\lambda_J|=O(|\lambda^{(k)}-\lambda_J|^3)$。证明思路（讲义原文）：若 $\|v^{(k)}-q_J\|\le\varepsilon$，Rayleigh 商误差是 $O(\varepsilon^2)$，再做一步反迭代得 $O(|\lambda^{(k)}-\lambda_J|\cdot\varepsilon)=O(\varepsilon^3)$。

## 14.5 复杂度

| 算法 | 每步代价 | 收敛阶 | 需要什么 |
|---|---|---|---|
| 幂法 | 1 次 matvec（$2\,\mathrm{nnz}$） | 线性，比值 $\lvert\lambda_2/\lambda_1\rvert$ | 只要 $A$ |
| 反迭代（固定 $\mu$） | 1 次三角/线性求解 | 线性，比值 $\lvert\mu-\lambda_J\rvert/\lvert\mu-\lambda_K\rvert$ | $A-\mu I$ 的分解 |
| RQI | 1 次线性求解（**每步都要重新分解**） | 三次 | $A-\lambda I$ 的分解，$O(n^3)$/步 |

RQI 每步 $O(n^3)$ 但只需 3–5 步；对**三对角**矩阵每步只要 $O(n)$，这是它能用于实际的原因〔教材〕。

## 14.6 实现陷阱

1. **初值恰好与 $q_1$ 正交**：幂法永远收敛不到 $\lambda_1$（$a_1=0$）。实践中用随机初值规避，但要检查。
2. **$|\lambda_1|=|\lambda_2|$（如 $\pm1$）**：幂法不收敛，向量在两个方向间振荡；需改用带位移的算法。
3. **反迭代的 $\mu$ 恰好等于特征值**：$A-\mu I$ 奇异，求解炸；实现时用 $\mu+\delta$ 微扰或 LDL 分解。
4. **RQI 收敛到"错的"特征值**：它收敛到离初值最近的（按 Rayleigh 商）特征值，不保证是最大或最小。
5. **在循环里跑 100 个初值仍每步做稠密 LU**：先归约到三对角（$O(n)$/步）再 RQI，否则 $100\times O(n^3)$ 不可接受。

# 第 15 章 特征值问题 III：QR 迭代与位移策略

> 对应材料：Lec 16 的两份 reading《The QR Algorithm I》《The QR Algorithm II》，以及 Lec 17（calendar："Shifted QR and Rayleigh quotients"）。

## 15.1 问题：同时求所有特征值

"纯"QR 算法简单得惊人〔讲义 reading I 第 9 页〕：

```
A^(0) = A
for k = 1,2,...:
    Q^(k) R^(k) = A^(k-1)        # QR 分解
    A^(k) = R^(k) Q^(k)          # 反序相乘
```

由于 $A^{(k)} = R^{(k)}Q^{(k)} = (Q^{(k)})^{\mathsf{T}} A^{(k-1)} Q^{(k)}$，每一步都是**正交相似变换**，特征值不变；在若干假设下 $A^{(k)}$ 收敛到 Schur 形（对称时对角化）。

## 15.2 为什么它等于"同时幂法"

这是本课最漂亮的一段等价性证明〔讲义 reading I 第 10–14 页〕。

**同时迭代**：取 $n$ 个线性无关初值组成 $V^{(0)}$，令 $V^{(k)} = A^k V^{(0)}$，则列空间收敛到前 $n$ 个特征向量张成的空间；但 $V^{(k)}$ 条件数爆炸，所以每步正交化：

```
Q^(0) = I
for k = 1,2,...:
    Z = A Q^(k-1)
    Q^(k) R^(k) = Z              # 约化 QR
    A^(k) = (Q^(k))^T A Q^(k)
```

**等价性**：两者生成同一列 $Q^{(k)} = \prod_{j\le k} Q^{(j)}$，且 $A^k = Q^{(k)} R^{(k)}$（$R^{(k)} = R^{(k)}\cdots R^{(1)}$），从而 $A^{(k)} = (Q^{(k)})^{\mathsf{T}} A Q^{(k)}$。于是 QR 迭代的收敛速度继承幂法：

$$\| q_j^{(k)} - (\pm q_j) \| = O(C^k), \qquad C = \max_{1\le j\le n} \left| \frac{\lambda_{j+1}}{\lambda_j} \right|$$

**第二重等价**（reading II 第 2 页）：$A^{-k}P = [Q^{(k)}P][P(R^{(k)})^{-\mathsf{T}}P]$ 是 $A^{-k}P$ 的 QR 分解（$P$ 为"翻转"置换矩阵），故 QR 迭代**同时**等价于对 $A^{-1}$ 做同时迭代——最后一列按反迭代演化。**一个算法同时是"最慢的幂法"和"最快的反迭代"**，这就是它无需初值就能求出全部特征值的原因。

## 15.3 位移：从"能收敛"到"能用"

纯 QR 的收敛因子是 $|\lambda_{j+1}/\lambda_j|$；若两个特征值模相近，收敛极慢。引入位移 $\mu_k$：

$$A^{(k-1)} - \mu_k I = Q^{(k)} R^{(k)}, \qquad A^{(k)} = R^{(k)} Q^{(k)} + \mu_k I$$

常用策略〔教材，Lec 17 主题〕：

| 策略 | $\mu_k$ | 效果 |
|---|---|---|
| Rayleigh 商位移 | $\mu_k = A^{(k)}_{nn}$（或当前块） | 对对称矩阵三次收敛 |
| Wilkinson 位移 | 右下 $2\times 2$ 块的两个特征值中较近者 | 保证实位移、全局稳健，LAPACK 默认 |
| 单/双位移 | 复共轭对用双步位移，保持实数运算 | 处理复特征值而不引入复数 |

**收缩（deflation）**：当次对角元 $|h_{n,n-1}| \le \epsilon(|h_{n-1,n-1}| + |h_{nn}|)$ 时把它置零，问题分成两块，规模递减。

## 15.4 复杂度

| 阶段 | 代价 |
|---|---|
| 归约到 Hessenberg | $\frac{10}{3}m^3$（第 13 章） |
| 每次带位移 QR 步（Hessenberg 上） | $O(m^2)$（$3m^2$ 量级） |
| 总迭代步数 | 典型每特征值 2–3 步，共 $O(m)$ 步 |
| 总体 | $O(m^3)$，常数约 $10m^3$–$25m^3$ |

**关键**：如果每步都在稠密 $A$ 上做，单步是 $O(m^3)$，总代价 $O(m^4)$——归约到 Hessenberg 把 $m^4$ 降到 $m^3$，这是"先归约再迭代"的全部价值。

## 15.5 实现陷阱

1. **不收缩就硬跑到最大迭代数**：次对角元已经小到 $10^{-18}$ 时仍在迭代，纯属浪费；必须在每步后检查并 deflate。
2. **位移用复数却用实数数组存**：$A - \mu I$ 变复数后 QR 分解代价翻倍；LAPACK 用双步位移（double-shift）规避。
3. **自己实现 QR 迭代**：这段代码极其难写对（收缩判据、位移选择、2×2 块处理），生产代码一律用 `schur` / `hessenberg` + `qr` 组合或直接 `eig`。
4. **对 Hessenberg 用 `np.linalg.qr` 每步重建 $Q$**：稠密 Householder 是 $O(m^3)$，无法利用 Hessenberg 结构；必须用 `scipy.linalg.lapack.dhseqr` 之类。另注意只有**上三角块**的对角元才是特征值，未收缩前的对角元只是近似。

# 第 16 章 Krylov 子空间 I：Arnoldi 与 Lanczos

> 对应材料：Lec 18（calendar："Krylov methods and the Arnoldi algorithm"）与 Lec 19 讲义的开篇概述（该讲义第 1 页对 Arnoldi 关系与代价给了精确表述）。

## 16.1 问题：$A$ 太大，不能分解

$n = 10^7$ 的稀疏矩阵：LU 分解产生的填充会炸到无法存储（第 20 章），$O(n^3)$ 更是不可能。但**乘一次 $A$ 很便宜**（$2\,\mathrm{nnz}$）。于是问题变成：**只用 matvec，能提取多少谱信息？**

答案是用 Krylov 子空间：

$$\mathcal{K}_n = \mathrm{span}\{b, Ab, A^2b, \ldots, A^{n-1}b\}$$

它由 $b$ 与 $A$ 的反复作用生成，天然只依赖 matvec。

## 16.2 Arnoldi 算法与 Arnoldi 关系

对 $\mathcal{K}_n$ 做 Gram–Schmidt 正交化（MGS 版本），得到 $Q_n = [q_1, \ldots, q_n]$ 与上 Hessenberg $H_n$，满足 **Arnoldi 关系**〔讲义 Lec 19 第 1 页〕：

$$AQ_n = Q_n H_n + r_n e_n^*, \qquad H_n = Q_n^* A Q_n, \quad r_n = h_{n+1,n} q_{n+1} \perp \mathcal{K}_n$$

```
q_1 = b/||b||
for j = 1..n:
    w = A q_j
    for i = 1..j:
        h_ij = q_i^* w
        w = w − h_ij q_i          # MGS
    h_{j+1,j} = ||w||
    if h_{j+1,j} == 0: break      # 不变子空间，精确终止
    q_{j+1} = w / h_{j+1,j}
```

**关键结构**：$H_n$ 是上 Hessenberg（$h_{ij} = 0$ 当 $i > j+1$）。这个"几乎上三角"的形状是后面一切（GMRES 的最小二乘、QR 迭代的位移、Lanczos 的三项递推）的根源。

## 16.3 Lanczos：对称情形的三项递推

$A = A^*$ 时 $H_n$ 是 Hermite 三对角，记作 $T_n$。此时 MGS 的内层循环只需保留前两个 $q$：

$$Aq_j = \beta_{j-1}q_{j-1} + \alpha_j q_j + \beta_j q_{j+1}$$

代价从 $O(mn^2)$（Arnoldi，每步与所有历史向量正交）降到 $O(mn)$（每步常数次 matvec + axpy）。这是 Lanczos 在对称问题上压倒性优势的来源。

## 16.4 Ritz 值与"幽灵特征值"

对 $H_n$ 做特征分解 $H_n z = \theta z$，则 $\theta$ 称为 **Ritz 值**，$Q_n z$ 称为 Ritz 向量，满足 $A(Q_n z) - \theta(Q_n z) \perp \mathcal{K}_n$——这是"在 Krylov 子空间里能做到的最好"。

**陷阱**：Lanczos 在浮点下会**丢失正交性**，导致同一个特征值被"发现"多次，出现**幽灵特征值（ghost eigenvalues）**〔讲义 Lec 19 第 1 页明确点名〕。解决办法：

1. 显式存储 $Q_n$ 并周期性**完全重正交化**（代价回到 $O(mn^2)$，但只在必要时做）；
2. 或选择性重正交化（只对"已收敛的 Ritz 向量"方向正交化）；
3. 或接受幽灵值，用"同一 Ritz 值出现多次"作为收敛信号（实践中最常见）。

## 16.5 误差与稳定性

- Arnoldi/Lanczos 本身是向后稳定的（$\|AQ_n - Q_n H_n\| = O(\epsilon\|A\|)$），但**正交性**会退化：Lanczos 的 $\|Q_n^* Q_n - I\|$ 可达 $O(\epsilon\kappa)$ 甚至更大；
- Ritz 值的收敛速度依赖谱分布：对"孤立的极端特征值"收敛快（$O((\lambda_2/\lambda_1)^k)$），对内部特征值几乎不收敛；
- **Ritz 值不是特征值的上界**（除非 $A$ 对称且用适当归一化），不能当"误差棒"用。

## 16.6 复杂度

| 算法 | 每步代价 | 总代价（$n$ 步） | 内存 |
|---|---|---|---|
| Arnoldi | $2\,\mathrm{nnz} + 4mn$ | $\Theta(mn^2)$〔讲义 Lec 19〕 | $mn$（存 $Q_n$） |
| Lanczos | $2\,\mathrm{nnz} + O(m)$ | $O(mn)$ | $mn$（若重正交化）/ $O(m)$（三项递推） |

## 16.7 实现陷阱

1. **不重正交化就跑 Lanczos 求重特征值**：会得到一串"假收敛"的重复 Ritz 值。
2. **$h_{j+1,j} = 0$ 时继续跑**：说明已找到不变子空间，应终止；强行除以 0 会产生 NaN。
3. **用 Arnoldi 求内部特征值**：Krylov 方法天然偏向极端特征值；求内部特征值要用 shift-invert（对 $A - \mu I$ 做 Arnoldi）。

4. **把 $H_n$ 当"小 $A$"直接 `eig`**：$n$ 稍大（几百）时 $O(n^3)$ 也不便宜，且数值上应利用 Hessenberg 结构。
5. **内存**：Arnoldi 存 $Q_n$ 需 $mn$ 个浮点数；$m=10^6, n=500$ 就是 4 GB，必须重启（第 19 章）。

---

# 第 17 章 Krylov 子空间 II：GMRES 及其收敛

> 对应材料：Lec 20（calendar："The GMRES algorithm and convergence of GMRES and Arnoldi"），OCW 无 PDF。本章据 Trefethen & Bau 的 "GMRES" 一讲与标准结论整理，标注 〔教材〕；数值实验为 〔实验〕。

## 17.1 问题：非对称线性系统

CG 只对对称正定有效。对一般 $A$，在 Krylov 子空间里最小化残差：

$$x_k = \arg\min_{x \in x_0 + \mathcal{K}_k} \|b - Ax\|_2$$

这就是 GMRES（Generalized Minimal RESidual）。它的名字就是它的定义。

## 17.2 用 Arnoldi 关系把问题降维

设 $x = x_0 + Q_k y$，则 $b - Ax = r_0 - AQ_k y = r_0 - Q_k H_k y - \underbrace{h_{k+1,k} q_{k+1} e_k^{\mathsf{T}} y}_{\perp \text{ 故可分离}}$，于是

$$\|b - Ax\|_2 = \|\beta e_1 - H_k y\|_2 + O(\text{常数})$$

其中 $\beta = \|r_0\|$。**$m \times m$ 的最小二乘变成 $k \times k$ 的最小二乘**，$k \ll m$。用 Givens 旋转增量地解这个小最小二乘，每步 $O(k)$ 而非 $O(k^3)$。

## 17.3 收敛界（以及它为什么几乎没用）

若 $A = X \Lambda X^{-1}$ 可对角化〔教材〕：

$$\frac{\|r_k\|}{\|r_0\|} \le \kappa_2(X) \min_{p \in \Pi_k,\, p(0)=1} \max_{\lambda \in \sigma(A)} |p(\lambda)|$$

**读法**：GMRES 的收敛由"谱点能否被一个过原点的 $k$ 次多项式压小"决定，前面的 $\kappa_2(X)$（特征向量矩阵的条件数）是"非正规性罚金"。这解释了为什么**特征值全在右半平面但矩阵高度非正规时 GMRES 会停滞**：谱看起来很好，$\kappa_2(X)$ 却是 $10^{10}$。

## 17.4 实测：重启的代价

取 1D 对流–扩散算子（$n = 100$，迎风格式，Péclet 数 $\mathrm{Pe}$ 控制非对称性），$b = \mathbf{1}$，tol $= 10^{-8}$〔实验〕：

| $\mathrm{Pe}$ | $\kappa_2(A)$ | 完整 GMRES 迭代数 | GMRES(20) 迭代数 |
|---|---|---|---|
| 0 | $4.1 \times 10^3$ | 100 | **1960** |
| 50 | $6.2 \times 10^2$ | 100 | 240 |
| 500 | $1.8 \times 10^2$ | 100 | 260 |

**最刺眼的一行**：对称情形下完整 GMRES 100 步收敛（$n = 100$，理论上 $n$ 步必收敛），而 GMRES(20) 需要 1960 步——**重启丢掉了 Krylov 空间里最有价值的方向**，收敛几乎停滞。这就是"能不全重启就不全重启"的定量理由。

## 17.5 复杂度

| 项目 | 代价 |
|---|---|
| 每步 | 1 次 matvec（$2\,\mathrm{nnz}$）+ $O(k)$ 正交化 + $O(k)$ Givens |
| 存储 | $O(mk)$（存全部 Krylov 基） |
| 完整 GMRES | 最多 $m$ 步，总 $O(m^2 n)$ 最坏 |
| GMRES($k$) | 每轮 $O(mk^2)$，总轮数不定 |

## 17.6 实现陷阱

1. **不重启导致内存爆**：$k$ 增大时 $Q_k$ 是 $m \times k$，$m = 10^7, k = 1000$ 就是 80 GB。
2. **重启太频繁**：如 17.4 所示，GMRES(20) 在对称问题上比完整 GMRES 慢 20 倍；重启长度应尽量大（如 100–300）或配合好的预条件子。
3. **忘记左/右预条件的区别**：左预条件改的是残差（收敛判据变成预条件后的残差），右预条件保留真实残差；诊断收敛时必须用**真实** $\|b - Ax\|$。
4. **用 GMRES 解 SPD 系统**：CG 每步只要 1 次 matvec + 常数向量操作，内存 $O(m)$；GMRES 每步 $O(k)$ 且存 $mk$。SPD 一律 CG。
5. **不检查 breakdown**：$h_{k+1,k} = 0$ 时 GMRES 已经得到精确解，继续算会除零。

---

# 第 18 章 Krylov 子空间 III：预处理与共轭梯度

> 对应材料：Lec 21（"Preconditioning techniques. The conjugate-gradient method"）与 Lec 22（"Convergence of conjugate gradient"）、Lec 23（"Biconjugate gradient algorithms"），OCW 均无 PDF。本章据 Trefethen & Bau 的 "Lanczos/CG" 与 "CG convergence" 两讲及标准结论整理，标注 〔教材〕。

## 18.1 问题：让 Krylov 方法"看得见"谱

Krylov 方法的收敛由谱条件数决定；预处理就是把 $A$ 换成 $M^{-1}A$（左预条件）、$AM^{-1}$（右预条件）或 $M^{-1/2}AM^{-1/2}$（对称预条件），使新矩阵的条件数接近 1，同时**保持 $M^{-1}v$ 便宜**。

## 18.2 共轭梯度：三项递推 + A-范数最优

对 $A \succ 0$，CG 在 $\mathcal{K}_k$ 中最小化**能量范数**误差 $\|e_k\|_A = \sqrt{e_k^{\mathsf{T}} A e_k}$：

```
x_0 = 0; r_0 = b; p_0 = r_0
for k = 0,1,2,...:
    α_k = (r_k^T r_k) / (p_k^T A p_k)
    x_{k+1} = x_k + α_k p_k
    r_{k+1} = r_k − α_k A p_k
    β_k = (r_{k+1}^T r_{k+1}) / (r_k^T r_k)
    p_{k+1} = r_{k+1} + β_k p_k
```

每步 **1 次 matvec + 2 次点积 + 3 次 axpy**，内存 $O(m)$——这是它统治 SPD 问题的原因。

## 18.3 收敛界

$$\frac{\|e_k\|_A}{\|e_0\|_A} \le 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k, \qquad \kappa = \kappa_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$$

由此得**迭代数估计**：

$$k \approx \tfrac{1}{2}\sqrt{\kappa}\,\ln\!\left(\frac{2}{\varepsilon}\right)$$

## 18.4 这个界有多松：一个实测

$n = 200$ 的 1D Laplacian（$\kappa = 1.637 \times 10^4$，$\sqrt{\kappa} \approx 128$），tol $= 10^{-10}$〔实验〕：

| 右端 | 实际迭代数 | 界的预测 |
|---|---|---|
| $b = \mathbf{1}$ | 100 | $k \approx 1530$（误差界），实际在第 100 步为 $2 \times 10^{-10}$ |
| $b \sim \mathcal{N}(0, 1)$ | 200（$= n$，精确算术下必达） | 同上 |

按公式，$k = 100$ 时界为 $2 \times 0.98449^{100} = 0.42$——**界说"误差还有 42%"，实际已经 $10^{-10}$**。原因：

1. 该界是**最坏情形**，对"特征值成簇"的矩阵极度保守；
2. $b = \mathbf{1}$ 只激发了约 $n/2$ 个奇模态（1D Laplacian 特征向量的对称性），有效维数减半；
3. CG 在收敛后期表现**超线性**（有限精度下 Krylov 子空间快速捕获极端特征向量）。

**实践结论**：$\sqrt{\kappa}$ 只用来判断"量级"（$\kappa = 10^4$ 时几百步，$\kappa = 10^{10}$ 时几万步），不要拿它当停机预测。

## 18.5 预处理子一览

| 预处理 | 构造代价 | 效果 | 适用 |
|---|---|---|---|
| Jacobi（对角） | $O(n)$ | 消掉量纲差异，$\kappa$ 常降 1–2 个量级 | 对角占优 |
| SSOR / SGS | $O(\mathbf{nnz})$ | 中等 | 一般稀疏 |
| 不完全 Cholesky IC(0)/ICT | $O(\mathbf{nnz})$ | 强，$\kappa$ 可降几个量级 | SPD、结构化 |
| ILU(k) / ILUT | $O(\mathbf{nnz})$ | 强，非对称 | 一般稀疏 |
| 代数多重网格（AMG） | 较贵 | 对 Poisson 型 $\kappa$ 与 $n$ 无关 | 椭圆 PDE |
| 直接法（$M = A$） | $O(n^3)$ | 一步收敛 | 小问题 |

## 18.6 非对称与不定：CG 的亲戚

| 算法 | 适用 | 代价 | 备注 |
|---|---|---|---|
| CG | SPD | 每步 1 matvec，$O(m)$ 内存 | 黄金标准 |
| CGNR/CGLS | 最小二乘（$A^{\mathsf{T}}A$ 或 $AA^{\mathsf{T}}$） | 每步 2 matvec | 条件数被平方 |
| BiCG / CGS | 一般非对称 | 每步 2 matvec，短递推 | 可能 breakdown，收敛不规则 |
| BiCGSTAB | 一般非对称 | 每步 2 matvec | 比 BiCG 平滑，常用 |
| GMRES | 一般非对称 | 每步 1 matvec，内存 $O(mk)$ | 最稳，但贵 |

## 18.7 实现陷阱

1. **对非对称矩阵用 CG**：理论保证全无，可能收敛到错误结果或停滞；先检查 `np.allclose(A, A.T)`。

2. **不设 `maxiter`**：CG 在有限精度下会"停滞"在某个残差水平，无限循环；必须设 `maxiter` 并用**相对**残差判据。
3. **预条件子构造失败不检查**：IC(0) 对某些 SPD 矩阵会遇到负主元（breakdown），需加对角移位（shift）。
4. **用残差判收敛而不用真实残差**：递推的 $r_k$ 会累积误差偏离真实 $b - Ax_k$；每 50–100 步重算一次真实残差。
5. **把 CG 用于最小二乘的正规方程**：$\kappa(A^{\mathsf{T}}A) = \kappa(A)^2$，迭代数按 $\kappa(A)$ 而非 $\sqrt{\kappa(A)}$ 增长——用 CGLS/LSQR 而非 CG。

---

# 第 19 章 重启的陷阱：为什么"朴素重启"不行

> 对应材料：Lec 19《Why restarting Arnoldi/Lanczos is not trivial》（S. G. Johnson，2019-03-21）。这是全课逻辑最密的一页讲义，值得逐行读。

## 19.1 问题：内存不够时，怎么"接着算"

Arnoldi 跑到第 $n$ 步时，$Q_n$ 占 $mn$ 内存、代价 $\Theta(mn^2)$ 也已难承受。自然的想法：**只保留 $k < n$ 个"最好的"方向，从那里继续**：

$$\hat{Q}_k = \underbrace{Q_n}_{m\times n}\ \underbrace{\hat{Q}}_{n\times k}, \qquad \hat{Q}^*\hat{Q} = I$$

问题是：$k$ 维子空间有无穷多个，随便挑一个会让 Arnoldi 的结构崩塌。

## 19.2 必须保住的性质

继续 Arnoldi 的前提是 Arnoldi 关系仍然成立〔讲义 Lec 19 第 1 页〕：

$$AQ_n = Q_nH_n + r_ne_n^*, \qquad H_n = Q_n^*AQ_n \text{ 上 Hessenberg}, \quad r_n \perp \mathcal{K}_n$$

重启后需要 $\hat{Q}_k$ 满足同样的形状：$A\hat{Q}_k = \hat{Q}_k\hat{H}_k + \hat{r}_ke_k^*$，$\hat{H}_k$ 上 Hessenberg、$\hat{r}_k \perp \hat{\mathcal{K}}_k$。**上 Hessenberg 性是 Lanczos 三项递推的命根子**，破坏了它，后面的步骤全部失效。

## 19.3 朴素重启为什么失败（讲义的核心推导）

最自然的挑法是取 **Ritz 向量**（当前最好的特征向量估计）：设 $H_nz = \nu z$，取前 $k$ 大的 $|\nu|$ 对应的 $Z$，做 QR：$\hat{Z} = \hat{Q}\hat{R}$，即 $\hat{Q} = \hat{Z}\hat{R}^{-1}$。代入：

$$\hat{H}_k = \hat{Q}^* H_n \hat{Q} = \hat{R}^{-1}\hat{Z}^* H_n \hat{Z}\hat{R} = \hat{R}^{-1}\hat{\Lambda}\hat{R} = \hat{R}\hat{\Lambda}\hat{R}^{-1}$$

（最后一步用了 $\hat{Z}$ 正交、$\hat{R}$ 上三角。）这是**三个上三角矩阵的乘积，仍是上三角**——比要求的"上 Hessenberg"还好，看起来完美。第二个交叉项也消失：$\hat{Q}_\perp^* H_n \hat{Q} = \hat{Q}_\perp^* \hat{Q}\hat{R}\hat{\Lambda}\hat{R}^{-1} = 0$。

**但第三项毁了这一切**〔讲义 Lec 19 第 3 页〕：

$$r_n e_n^* \hat{Q} = r_n \cdot (\hat{Q} \text{ 的最后一行})$$

虽然 $r_n \perp \mathcal{K}_n$ 保证 $r_n \perp \hat{\mathcal{K}}_k$（即 $\hat{Q}_k^* r_n = 0$），但**$\hat{Q}$ 的最后一行一般不是 $e_k^*$ 的倍数**。于是重启后的矩阵多出一个"满行"的残差项，Hessenberg 结构被破坏。Lanczos 情形同样失败：$\hat{H}_k = \hat{\Lambda}$ 是对角（更好），但最后一行仍然不是 $e_k^*$。

> **一句话**：朴素重启"做得太好"——$\hat{H}_k$ 成了上三角而不是上 Hessenberg，$r_n$ 正交于整个 $\mathcal{K}_n$ 而不只是 $\hat{\mathcal{K}}_k$。正是这份"过好"占掉了自由度，使得最后一行无法成为 $e_k^*$ 的倍数。

## 19.4 正确做法：隐式重启

讲义给出的解法是：**不要取 $H_n$ 的特征向量，而取对 $H_n$ 做 $n-k$ 步带位移 QR 迭代后得到的 Schur 向量**〔讲义 Lec 19 第 3 页〕。这样做的估计略"差"（只到上 Hessenberg 而非上三角），恰好留出足够自由度把最后一行调成 $e_k^*$ 的倍数。完整证明需要"careful and tedious calculation"，讲义明确说"你几乎不需要知道细节"——实践里用 ARPACK 的 `eigs` / `svds` 即可。

## 19.5 复杂度

| 方案 | 每轮代价 | 内存 |
|---|---|---|
| 不重启 | — | $O(mn)$，$n$ 受内存限制 |
| 朴素重启 | $O(nk^2)$ + 一次 QR | $O(mk)$，但**结果错误** |
| 隐式重启（IRAM） | $O(nk^2)$ + $n-k$ 步位移 QR | $O(mk)$，正确 |

## 19.6 实现陷阱

1. **自己写"取前 $k$ 个 Ritz 向量重启"**：这正是讲义点名要抵制的诱惑（"please resist the temptation to do naive restarting!"）。
2. **对 Lanczos 用"截断 $Q_n$"重启**：Lanczos 的三项递推依赖严格的三对角结构，截断后立刻失效。
3. **重启长度取太小**：$k < 2p$（$p$ 为想要的特征值个数）时隐式重启也无法收敛。
4. **不用现成实现**：ARPACK（`scipy.sparse.linalg.eigs`）、SLEPc、Julia 的 `Arpack` 都是 IRAM 的成熟实现。

# 第 20 章 稀疏矩阵与稀疏直接法

> 对应材料：Lec 24《Sparse Matrix Algorithms》（Per-Olof Persson 署名，2006-11-28）。

## 20.1 问题：零也是要花钱的

"稀疏矩阵是有足够多的零、值得利用的矩阵"（Wilkinson 语，讲义开篇引用）；"结构化矩阵是有足够结构、值得利用的矩阵"（如 Toeplitz）；"稠密矩阵两者都不是"〔讲义 Lec 24 第 2 页〕。稀疏算法的两条设计原则：

1. **存储** $O(\mathbf{nnz})$；
2. **时间接近** $O(\mathbf{flops})$。

## 20.2 存储：压缩列（CSC）

讲义给的 CSC 三数组表示〔讲义 Lec 24 第 4 页〕：`Pr`（非零值，长度 nnz）、`Ir`（行号，长度 nnz）、`Jc`（列起始指针，长度 n+1）。内存约为 $\mathbf{1.5\,nnz} + \mathbf{0.5}n$（按 double + int 计）。

| 操作 | 代价 |
|---|---|
| 按列遍历 | $O(\mathbf{nnz})$ |
| 单元素查找 | $O(\log(\text{该列元素数}))$（二分） |
| 插入新非零 | 很贵（要挪动整列） |
| 稀疏矩阵一向量乘 | $O(\mathbf{nnz})$ |

**推论**：稀疏矩阵应当**先组装后使用**（COO→CSR/CSC），不要在循环里逐个插入。

## 20.3 填充（fill-in）：稀疏直接法的核心困难

对 $A = LU$ 或 $A = LL^{\mathsf{T}}$，消元会产生新的非零——**填充**。图论模型〔讲义 Lec 24 第 5–6 页〕：

- 图 $G(A)$：顶点 = 行列指标，边 = 非零元；
- 消去顶点 $j$ 时，把 $j$ 的所有**更高编号邻居**两两连边（形成团）；
- 填充 = 新增的边。

**2D 模型问题**（$n \times n$ 网格，$N = n^2$ 个未知数，五点差分）的经典结论〔讲义 Lec 24 第 7 页〕：

| 排序 | 填充量 |
| --- | --- |
| 自然排序（行优先） | $O(N^{3/2})$ |
| 任意排序的下界 | $\Omega(N \log N)$ |
| 嵌套剖分（nested dissection） | $O(N \log N)$（渐进最优） |

## 20.4 排序策略

| 策略 | 思想 | 实践 |
| --- | --- | --- |
| 带宽/包络（RCM、Sloan） | 让非零靠近对角线 | "细长"问题常胜 |
| 最小度（AMD/COLAMD） | 每次消去度数最小的顶点 | 中等规模常胜，实现难（近似最小度是当前冠军：Amestoy—Davis—Duff） |
| 嵌套剖分（ND） | 递归找分隔集，最后编号 | 超大规模常胜；现代通用排序多是 ND/MD 混合 |

MATLAB 对应函数〔讲义 Lec 24 第 11 页〕：`symrcm`（RCM）、`symamd`（对称近似最小度）、`colamd`（非对称）、嵌套剖分需 MESHPART 工具箱。Python 里对应 `scipy.sparse.csgraph.reverse_cuthill_mckee` 与 `scipy.sparse.linalg.splu(..., permc_spec=...)`。

## 20.5 直接法的复杂度（讲义原表）

对"形状良好"的有限元网格，$N$ 个节点〔讲义 Lec 24 第 12 页〕：

| | 1D | 2D | 3D |
| --- | --- | --- | --- |
| 空间（填充） | $O(N)$ | $O(N \log N)$ | $O(N^{4/3})$ |
| 时间（flop） | $O(N)$ | $O(N^{3/2})$ | $O(N^2)$ |

**读法**：3D 问题的稀疏 Cholesky 是 $O(N^2)$，这就是为什么 3D 大规模问题必须转向迭代法（第 16—18 章）或多重网格。

## 20.6 实现陷阱

1. **用 `np.zeros((n,n))` 存稀疏矩阵**：$n = 10^5$ 时 80 GB；用 `scipy.sparse.csr_matrix`。
2. **用 `A[i,j] += v` 逐个组装 CSR**：每次插入 $O(\mathrm{nnz})$，总 $O(\mathrm{nnz}^2)$；用 COO 三元组一次性 `tocsc()`。
3. **`scipy.sparse.linalg.spsolve` 对 3D 问题**：会因填充耗尽内存；应改用 CG/GMRES + 预条件。

4. **忽视排序**：同一个矩阵，自然排序与 AMD 排序的分解时间可差 10–100 倍〔推断：依矩阵结构〕。
5. `A.T @ A` **生成稀疏矩阵**：稀疏矩阵乘稀疏矩阵会产生大量填充，$A^{\mathsf{T}}A$ 的 nnz 常是 $A$ 的几十倍；正规方程在稀疏情形更不可取。

---

# 第 21 章 内存层次、缓存与分块算法

> 对应材料：Lec 11《Performance Experiments with Matrix Multiplication》与 Lec 12《Experiments with Cache-Oblivious Matrix Multiplication》（均为 S. G. Johnson，实验平台 2.66 GHz Intel Core 2 Duo、gcc 4.1.2、ATLAS 3.6.0）。

## 21.1 问题：flop 数不能预测时间

一个三重循环的矩阵乘法 $C = AB$（$m \times n$ 乘 $n \times p$）是 $2mnp$ flop〔讲义 Lec 11 第 2 页〕。但讲义实测：**同样的 flop 数，速度差 10 倍**，且曲线上出现台阶——台阶正好对应"工作集超过 L1/L2 缓存"。

讲义的三条结论〔讲义 Lec 11 第 6 页〕：

1. 不理解内存效率就无法理解性能，**缓存比 flop 数重要约 10 倍**；
2. 计算机比你想的复杂；
3. 再简单的算法也难以写好：10 行矩阵乘法 → ATLAS 的 130,000+ 行。

## 21.2 理想缓存模型与缓存复杂度

Ideal-cache 模型（讲义另附 *Ideal-Cache Terminology* 一页）：缓存分 $Z$ 个字节、每行 $L$ 字节，高度自治（omniscient）地做最优替换。在此模型下：

| 算法 | 缓存复杂度（$n \times n$ 乘 $n \times n$） |
|---|---|
| 三重循环（朴素） | $\Theta(n^3)$（每个 flop 一次 miss） |
| 分块（blocked，块大小 $\Theta(\sqrt{Z})$） | $\Theta\left(\frac{n^3}{L\sqrt{Z}}\right)$ 次 cache miss |
| 递归分治（cache-oblivious） | $\Theta\left(\frac{n^3}{L\sqrt{Z}}\right)$（同样最优，且**不需知道 $Z$**；讲义简写为 $\Theta(n^3/\sqrt{Z})$） |

**缓存无关（cache-oblivious）** 的美妙之处：算法不知道 $Z, L$，却对所有层级同时最优。代价是常数大、递归开销高。

## 21.3 讲义给的 25 行实现

```
/* C = C + A*B，行主序；当 m+n+p <= 48 时退化为三重循环 */
void add_matmul_rec(const double *A, const double *B, double *C,
                    int m, int n, int p, int fdA, int fdB, int fdC) {
    if (m+n+p <= 48) {                          /* 基例：约 16x16 */
        for (i...) for (k...) { s = 0; for (j...) s += A[..]*B[..]; C[..] += s; }
    } else {                                    /* 把 C 切成 8 块，递归 8 次 */
        add_matmul_rec(A,    B,      C,   m2, n2, p2, ...);
        add_matmul_rec(A+n2, B+n2*fdB, C, m2, n-n2, p2, ...);   /* 其余 6 块同理 */
    }
}
```

讲义特别提示〔讲义 Lec 12 第 3 页〕：**基例不能递归到 $1 \times 1$**（每个元素一次函数调用、无寄存器复用，性能崩）；基例取 $\approx 16 \times 16$ 是实践折中。

## 21.4 但缓存无关还不够：寄存器也是缓存

讲义实测发现：消除了缓存台阶后，绝对性能仍差一倍〔讲义 Lec 12 第 5–6 页〕。原因是**寄存器（约 100 个）构成一个极小但极快的缓存**，而三重循环没有利用它。要喂饱寄存器必须：

- 把块加载到局部变量（=寄存器），做**展开的**矩阵乘；
- 展开的 $n \times n$ 乘法有 $(n^3)!$ 种代码顺序，编译器找不到最优调度（NP 难）；
- 所以 ATLAS 用**自动生成 + 实测搜索**来产出大量硬编码基例。

## 21.5 实践数字（讲义平台，2008 年）

| 实现 | 相对性能 |
|---|---|
| 朴素三重循环（$n$ 小，全在 L1） | 接近峰值 |
| 朴素三重循环（$n$ 超过 L2） | 掉到峰值的 1/10 |
| 缓存无关递归 | 无台阶，但绝对值仍低 |
| ATLAS `dgemm` | 接近峰值（约 2 flop/cycle，SSE2） |

## 21.6 实现陷阱

1. **循环顺序随意**：$ijk$ 与 $ikj$（$C$ 行主序时）差 5–10 倍；内层循环应沿**连续内存**方向。
2. `for i: for j: C[i,j] = dot(A[i,:], B[:,j])`：这是"点积形式"，对 $B$ 是列访问，缓存极差；应写"axpy 形式"或直接调 BLAS。

3. **自己写矩阵乘**：NumPy 的 `A@B` 走 OpenBLAS/MKL，已分块 + 汇编优化，手写循环几乎不可能更快。
4. **相信 flop 数做性能预测**：同一算法在不同缓存配置下差 10 倍；必须实测（但要小心计时噪声，见大纲对 final project 的提醒："timing measurements are more about implementation quality than algorithms"）。
5. **忽略 NUMA**：多路服务器跨 socket 访问内存延迟翻倍，分块应按 NUMA 域做〔推断〕。

---

# 第 22 章 FFT：Cooley–Tukey 与快速变换家族

> 对应材料：Lec 38《Fast Fourier Transform Algorithms》（S. G. Johnson，MIT IAP 2008 讲义，2019 课程复用）与 Lec 39《Fast Fourier Transform and FFTW》（幻灯片）。

## 22.1 问题：$O(N^2)$ 的 DFT

DFT 与其逆〔讲义 Lec 38 式 (1)(2)〕：

$$y_k = \sum_{n=0}^{N-1} x_n e^{-\frac{2\pi i}{N}nk}, \qquad x_n = \frac{1}{N}\sum_{k=0}^{N-1} y_k e^{+\frac{2\pi i}{N}nk}$$

直接算每个 $y_k$ 要 $N$ 次乘加，共 $O(N^2)$。FFT 把它降到 $O(N \log N)$——讲义称其为"20 世纪最重要的十个算法之一"，并列举应用：解 PDE、数字信号处理（MP3 压缩）、大数乘法（算 $\pi$ 到 $10^{12}$ 位）。

## 22.2 Cooley–Tukey：$N = N_1 N_2$ 的指标重排

把输入写成二维 $N_1 \times N_2$：$n = N_1 n_2 + n_1$；输出写成转置的 $N_2 \times N_1$：$k = N_2 k_1 + k_2$。代入得〔讲义 Lec 38 式 (3)〕：

$$y_{N_2 k_1 + k_2} = \sum_{n_1=0}^{N_1-1} e^{-\frac{2\pi i}{N} n_1 k_2} \left[ \sum_{n_2=0}^{N_2-1} e^{-\frac{2\pi i}{N_2} n_2 k_2} x_{N_1 n_2 + n_1} \right] e^{-\frac{2\pi i}{N_1} n_1 k_1}$$

三层结构一目了然：

1. 内层 $[\cdot]$：对每个 $n_1$ 做**长度 $N_2$ 的 DFT**；
2. 中间乘 **twiddle factor（旋转因子）** $e^{-\frac{2\pi i}{N} n_1 k_2}$；
3. 外层：对每个 $k_2$ 做**长度 $N_1$ 的 DFT**。

递归下去，若 $N$ 的因子都有界（如 $N = 2^m$），总代价 $O(N \log N)$。**关键洞察**：$y_k = x(\omega_N^k)$，即"多项式 $x(z)$ 在 $N$ 次单位根处的取值"，而

$$x(z) \bmod (z^N - 1) \text{ 可以递归地通过因式分解 } z^N - 1 \text{ 求得}$$

radix-2 对应 $z^N - a = (z^{N/2} - \sqrt{a})(z^{N/2} + \sqrt{a})$ 的递归分解〔讲义 Lec 38〕。

## 22.3 术语与算法家族

| 术语 | 含义 |
|---|---|
| radix（基） | 分解中较小的那个因子 $r$ |
| DIT（decimation in time） | $N_1 = r$，先对时间下标分奇偶 |
| DIF（decimation in frequency） | $N_2 = r$，先对频率下标分奇偶 |
| butterfly（蝶形） | 长度 $r$ 的小 DFT 基本单元 |
| mixed-radix | 不同层用不同基 |

讲义列出的"其他家族"〔讲义 Lec 38 第 2 页 / Lec 39〕：

- **prime-factor algorithm**（1958）：$N = N_1 N_2$ 且 $\gcd(N_1, N_2) = 1$ 时，用中国剩余定理重排指标，**没有 twiddle 因子**；
- **Rader 算法**（1968）：$N$ 为素数时，用乘法群生成元把 DFT 变成长度 $N-1$ 的循环卷积，再用（非素数长度）FFT 算；
- **Bluestein 算法**：任意 $N$，用 $nk = \frac{n^2 + k^2 - (n-k)^2}{2}$ 把 DFT 变成卷积，零填充后用 FFT；
- **specialized transforms**：实输入（`rfft`）与 DCT/DST 等实对称变体。

## 22.4 复杂度与"常数之战"

| 算法 | flop（实数） |
|---|---|
| 直接 DFT | $O(N^2)$（$N = 1024$：约 $10^6$） |
| radix-2 FFT | $\approx 5N \log_2 N$（$N = 1024$：约 $5.1 \times 10^4$） |
| split-radix（Yavne 1968） | 比 radix-2 省约 20% |
| 2007 年后的改进 | 再省约 6% |

〔讲义 Lec 39〕讲到：1979 年 Winograd 证明乘法数可达 $\Theta(n)$，但加法太多；**过去 15 年 flop 数已不再决定速度（差异只有 ±20%），而实际运行时间可差 10 倍以上**——瓶颈又回到了内存层次（第 21 章）。这也解释了 FFTW 的设计哲学：不只追求最少 flop，而是**用"codelet + plan 搜索"适配具体机器的缓存与寄存器**。

**$\Theta(N \log N)$ 是不是下界？** 讲义明确说：**未证明**（"the proof of this remains an open problem"）。这是本课程里罕见的"公开问题"级陈述，值得记住。

## 22.5 实测

$N = 16$ 时用 DFT 矩阵 $F_{kn} = e^{-2\pi ikn/N}$ 直接乘与 `np.fft.fft` 对比，最大绝对误差 $2.0 \times 10^{-14}$〔实验〕——两种算法在浮点下一致到 $O(\epsilon)$，说明 FFT 的指标重排没有引入额外误差放大。

## 22.6 实现陷阱

1. **长度不是小因子乘积**：$N$ 为大素数时 Cooley–Tukey 无能为力（退化为 Rader/Bluestein，或 $O(N^2)$）；实践中把 $N$ 补零到 2 的幂或"高度合数"（如 $2^a3^b5^c$）。
2. **补零改变结果**：补零做 FFT 得到的是**更密的频谱采样**，不是"更高分辨率的频谱"——分辨率由原始观测时长决定。
3. **忘记 `fftfreq` 的顺序**：`np.fft.fft` 的输出是 $[0, \dots, N/2, -N/2, \dots, -1]$ 顺序，画图前要 `fftshift`。
4. **实输入用 `fft` 而非 `rfft`**：浪费一半计算与内存；`rfft` 只算非负频率。
5. **归一化约定混乱**：NumPy 正变换不除 $N$、逆变换除 $N$；MATLAB 相同；但有些库（如 FFTW 的某些封装）两边都除 $\sqrt{N}$。跨库对比必须查清。
6. **多维 FFT 的轴顺序**：`np.fft.fft2` 默认对最后两轴，图像处理里常需 `axes=(0,1)`。

---

# 第 23 章 数值积分：梯形公式的"救赎"

> 对应材料：Lec 33a《Numerical integration and the redemption of the trapezoidal rule》（S. G. Johnson，IAP 2011–01–06）与 Lec 33b《Fourier Cosine Series Examples》。

## 23.1 问题：$O(1/N^2)$ 还是指数收敛？

复合梯形公式（$N$ 个子区间，$h = \pi/N$）〔讲义 Lec 33a 式 (2.1)〕：

$$I_N = \frac{\pi}{N}\left[\frac{f(0)+f(\pi)}{2} + \sum_{n=1}^{N-1} f(n\pi/N)\right]$$

**粗糙估计**：每段局部误差 $\sim h^3 f''$，共 $N$ 段，总误差 $\sim h^2 = 1/N^2$。这是绝大多数教材停下的地方。

## 23.2 讲义的两个实验

取 $f_1(x) = e^{\sin[(x+1)^2 + 2\cos(4x+1)]}$：误差随 $N$ 严格按 $1/N^2$ 下降（双对数坐标下斜率 $-2$）〔讲义 Lec 33a 图 3 左〕。

取 $f_2(x) = e^{\sin[\sin^2(x+1) + 2\cos(4x+1)]}$：误差在**半对数**坐标下成直线——**指数收敛**。区别只有一点：$f_2$ 是**周期**的（$f(x+\pi) = f(x)$）。

## 23.3 用 Fourier 余弦级数解释"奇迹"

把 $f$ 展成 $f(x) = \frac{a_0}{2} + \sum_{k\ge 1} a_k \cos kx$，逐项代入梯形公式。关键恒等式〔讲义 Lec 33a 第 7 页〕：

$$S_k = \frac{\cos(0) + \cos(k\pi)}{2} + \sum_{n=1}^{N-1} \cos(nk\pi/N) = \begin{cases} N, & k = 2mN \\ 0, & \text{其他} \end{cases}$$

于是

$$I_N = I + \pi \sum_{m=1}^{\infty} a_{2mN}$$

**这是精确等式**，不是估计。读法：

- 梯形公式"杀死"了所有 $k < 2N$ 的频率分量，只漏掉 $k \ge 2N$ 的**混叠**项；
- 若 $f$ 光滑，$a_k$ 指数衰减，故误差 $\sim a_{2N}$ **指数小**；
- 若 $f$ 不光滑（$a_k$ 只多项式衰减），误差退化到 $1/N^2$ 或更差。

**这就是"梯形公式的救赎"**：同一个公式，在周期光滑函数上是指数收敛的谱方法，在一般函数上只是二阶。

## 23.4 从梯形到 Clenshaw–Curtis

把 $x = \cos\theta$ 代入，则 $[0, \pi]$ 上的梯形公式变成 $[-1, 1]$ 上的 **Clenshaw–Curtis 求积**（在 Chebyshev 点上采样，权重由 cos 变换给出）。讲义说这一步把"最粗糙的公式"变成"最好的公式之一"〔讲义 Lec 33a 摘要〕。进一步用 Gauss 求积（$N$ 个点可精确积分 $2N-1$ 次多项式）可再省一半点数。

## 23.5 复杂度

| 方法 | 误差阶 | 点数 | 备注 |
|---|---|---|---|
| 复合梯形 | $O(h^2)$（一般）/ 指数（周期光滑） | $N+1$ | 最简单，可复用旧点（加倍时只算新点） |
| Simpson | $O(h^4)$ | $2N+1$ | 要求 $f \in C^4$ |

| 方法 | 误差阶 | 点数 | 备注 |
| --- | --- | --- | --- |
| 高阶 Newton–Cotes | 可能**发散**（Runge 现象） | 等距点 | 不要用 $n > 7$ 的等距公式 |
| Clenshaw–Curtis | 谱精度（光滑） | $N+1$，Chebyshev 点 | 点可复用，权重有快速算法 |
| Gauss–Legendre | 谱精度，$2N-1$ 次精确 | $N$ | 点不可复用（换 $N$ 要全重算） |
| 自适应（Gauss–Kronrod） | 局部谱精度 | 动态 | `scipy.integrate.quad` |

## 23.6 实现陷阱

1. **等距点用高阶 Newton–Cotes**：$n \geq 10$ 时权重出现巨大正负交替，Runge 现象让误差随 $n$ **增大**。
2. **对端点奇异的函数用梯形**：如 $\int_0^1 x^{-1/2}dx$，误差退化到 $O(N^{-1/2})$；应换变量或用以端点奇异性加权的 Gauss 公式。
3. **高维用张量积网格**：$d$ 维、每维 $N$ 点需 $N^d$ 次求值；$d=10, N=20$ 就是 $10^{13}$ 次。改用 Monte Carlo（$O(\varepsilon^{-2})$ 与 $d$ 无关）或稀疏网格。
4. **忽略求积的"条件数"**：$\sum|w_i|/|\sum w_i|$ 大时（如高阶 Newton–Cotes），函数值的舍入误差被放大。
5. **自适应求积不设最大深度**：在不可积奇点附近会无限细分。

---

# 第 24 章 优化与伴随方法：从 BFGS 到反向传播

> 对应材料：Lec 25《A Brief Overview of Optimization Problems》、Lec 26《Adjoint Methods》、Lec 27《Adjoint methods and sensitivity analysis for recurrence relations》、Lec 30《Quasi-Newton optimization》，均为 S. G. Johnson 讲义。

## 24.1 问题：为什么数值线性代数课要讲优化

因为优化的每一步几乎都是一次线性求解：Newton 法解 $Hd = -g$，拟 Newton 维护 $H$ 的近似，最小二乘是 Gauss–Newton 的核心。反过来，优化给数值线性代数提供了最大的应用场景（训练、反演、最优控制）。

## 24.2 拟 Newton 与 BFGS

Newton 法要 Hessian $H$，代价 $O(n^2)$ 存储 + $O(n^3)$ 分解；拟 Newton 只用梯度差维护 $H$ 的近似，每步 $O(n^2)$。BFGS 更新〔讲义 Lec 30〕：

$$B_{k+1} = B_k + \frac{y_k y_k^{\mathsf{T}}}{y_k^{\mathsf{T}} s_k} - \frac{B_k s_k s_k^{\mathsf{T}} B_k}{s_k^{\mathsf{T}} B_k s_k}, \qquad s_k = x_{k+1} - x_k,\ y_k = \nabla f_{k+1} - \nabla f_k$$

它满足割线条件 $B_{k+1} s_k = y_k$，且在所有满足割线条件的对称更新中是"最小改变"的（变分刻画）。L–BFGS 只存最近 $m$ 对 $(s, y)$，内存 $O(mn)$——这是大规模优化的默认选择。

## 24.3 伴随方法：一次反向传播换全部梯度

设 $f(x) = g(y(x))$，$y \in \mathbb{R}^m$ 由前向模型给出，$x \in \mathbb{R}^n$（$n$ 可以极大）。**有限差分**要 $n+1$ 次前向；**伴随方法**只要 1 次前向 + 1 次反向〔讲义 Lec 26〕：

$$\nabla_x f = \left(\frac{\partial y}{\partial x}\right)^{\mathsf{T}} \underbrace{\frac{\partial g}{\partial y}}_{\lambda}$$

其中 $\lambda$ 由**反向**递推求得（解一次转置线性系统）。代价与 $n$ 无关——这正是深度学习里反向传播（backprop）的数学内容，也是 Lec 27 讨论"递推关系的灵敏度"时的核心。

| 方法 | 代价 | 何时用 |
|---|---|---|
| 前向差分 | $n+1$ 次前向 | $n$ 很小 |
| 前向模式 AD | $\propto n$ 倍前向 | 输入维小、输出维大 |
| 反向模式 AD / 伴随 | $\propto m$ 倍前向 | 输出维小（标量损失）、输入维大 |
| 复数步长差分 | 1 次前向（精度 $O(\epsilon)$ 而非 $O(\sqrt{\epsilon})$） | 无法改代码时的黑盒梯度 |

## 24.4 复杂度

| 算法 | 每步代价 | 收敛 |
|---|---|---|
| 梯度下降 | $O(n)$ + 1 次梯度 | 线性，条件数依赖 |
| BFGS | $O(n^2)$ | 超线性 |
| L–BFGS | $O(mn)$ | 超线性（实践中接近 BFGS） |
| Newton | $O(n^3)$（解 $Hd = -g$） | 二次（局部） |
| 伴随梯度 | 1 次前向 + 1 次反向 | —（是求梯度，不是迭代） |

## 24.5 实现陷阱

1. **用有限差分验证梯度时步长取 $10^{-8}$**：截断误差 $O(h)$ 与舍入误差 $O(\epsilon/h)$ 的平衡点在 $h \approx \sqrt{\epsilon} \approx 10^{-8}$，但此时相对误差已有 $10^{-8}$；用**复数步长**（$h = 10^{-20}$）可得机器精度梯度〔教材/常识〕。
2. **BFGS 不检查曲率条件**：$y^{\mathsf{T}}s \le 0$ 时更新会破坏正定性，需跳过更新或做阻尼。
3. **伴随法存全部中间量**：反向需要前向的中间结果，内存 $\propto$ 步数；用 checkpointing（重算）换内存。
4. **把伴随当"近似梯度"**：伴随给的是**精确**梯度（到浮点精度），不是近似；把它和有限差分混淆会导致调试方向错误。
5. **忽略优化问题的条件数**：Hessian 的条件数决定梯度下降的收敛速度，与第 5 章的 $\kappa$ 是同一件事；预条件 = 变量替换。

---

# 第 25 章 与已有笔记的衔接

## 25.1 与 18.06 线性代数笔记

| 18.06 笔记里的位置 | 18.335 的补丁 | 本笔记对应 |
|---|---|---|
| "矩阵分解速查表"中 $PA=LU$、$A=QR$、$A=U\Sigma V^{\mathsf{T}}$ 的存在性 | 存在性不等于可算性；每条分解都要问"浮点下误差多大、多少 flop" | 第 7–12 章 |
| 正交投影 $P=QQ^{\mathsf{T}}$ | $Q$ 只有在 $O(\kappa\epsilon)$ 内才正交；CGS 造的 $Q$ 可能完全失效 | 7.3 的实测表 |
| 最小二乘 = 正规方程 | $\kappa(A^{\mathsf{T}}A)=\kappa(A)^2$，正规方程在大 $\kappa$ 下报废 | 11.3 的 Hilbert 表 |
| 特征值 = $\det(A-\lambda I)=0$ | 特征多项式方法是数值上最差的方法；特征值问题的条件数是 $1/\lvert y^*x\rvert$ | 第 13 章 |
| SVD 的几何意义 | 算 SVD 要 $O(mn^2)$；不要通过 $A^{\mathsf{T}}A$ 算 | 12.4 |
| 正交矩阵 $Q^{\mathsf{T}}Q=I$ | 浮点里是 $\lVert Q^{\mathsf{T}}Q-I\rVert=O(n\epsilon)$（Householder）到 $O(\kappa^2\epsilon)$（CGS） | 8.3、7.3 |

**一句话**：18.06 的定理全都对，18.335 补的是"折扣率"。

## 25.2 与 18.100A 实分析笔记

18.100A 提供了本课程**唯一**的"非代数"工具：

1. **紧性与极值定理** → 范数等价性证明的第 4 步（第 4.2 节）。没有紧性，$\min_{\|u\|_1=1}\|u\|_a$ 只是下确界，可能取不到，等价性证明就断了。
2. **单调有界收敛** → Newton 法 $\sqrt{a}$ 的收敛性证明（第 6.2 节，讲义明确引 Rudin 定理 3.14）。
3. **泰勒展开与 $O(\cdot)$ 记号** → 所有收敛阶的推导（$\delta_{n+1}=\delta_n^2/2$）。
4. **极限与连续性** → 浮点算术的"极限语义"：$4-\infty=\lim_{x\to\infty}(4-x)=-\infty$ 这类定义（讲义 Lec 2 第 4 页）。

反过来，18.335 给 18.100A 的"$\varepsilon$–$\delta$"提供了工程含义：$\epsilon_{\text{machine}}$ 就是这台机器上能取到的最小 $\varepsilon$。

## 25.3 与 Stanford EE364a 凸优化笔记

| EE364a 的概念 | 18.335 的视角 |
|---|---|
| 内点法每步解 $H\Delta x=-g$ | $H$ 的条件数决定能解出几位；$\kappa(H)=10^{12}$ 时双精度只剩 4 位 |
| 稀疏 KKT 系统 | 第 20 章的填充与排序直接决定内点法能否跑 |
| 拟 Newton（BFGS/L–BFGS） | 第 24 章；L–BFGS 的两循环递推是 $O(mn)$ 的关键 |
| 梯度/KKT 残差作为停机判据 | 残差判据与"真实误差"的区别，同 18.7 的 CG 陷阱 |
| 问题 scaling（变量替换） | 就是预条件（第 18.5 节）：把 $\kappa$ 降下来 |

**互补关系**：EE364a 关心"怎么建模成凸问题"，18.335 关心"模型给了之后，这个线性系统能不能解、解得多准"。

## 25.4 与 CS336 大模型笔记（数值稳定）

CS336 笔记里散落的每一条"训练不稳定"，都能在 18.335 找到根：

| CS336 现象 | 18.335 解释 | 对应章节 |
|---|---|---|
| fp16 下 $10^{-8}$ 塌成 0 | 非规格化/下溢：fp16 最小规格化数 $6\times10^{-5}$，最小非规格化 $6\times10^{-8}$ | 2.2 |
| 混合精度：参数 bf16、优化器状态 fp32 | 优化器状态要跨 $10^4$–$10^6$ 步累积，等效于长求和，需要 $\sqrt{n}\epsilon$ 而非 $\epsilon$ 的精度 | 3.3、2.5 |
| Adam 的 $\epsilon=10^{-8}$ | 防止 $v_t\to0$ 时 $1/\sqrt{v_t}$ 爆炸——本质是"除以接近 0 的量"的经典陷阱 | 2.5 陷阱 3 |
| softmax 要先减最大值 | 防 $e^x$ 上溢（$x>88$ 时 fp32 溢出）；这是"重排公式以避免中间量溢出"的标准手法 | 2.5、23.6 |
| 梯度爆炸/消失 | 连乘的误差/数值放大，等价于 $\prod\|J_i\|$ 的指数增长/衰减 | 14.3（幂法的比值） |

| CS336 现象 | 18.335 解释 | 对应章节 |
|---|---|---|
| loss scaling | 把梯度整体放大再缩小，等价于"换单位"以避免下溢 | 2.2 |
| FlashAttention 的在线 softmax | 分块 + 递推的"滑动最大值"，与 3.4 的成对求和同属"重排计算顺序换数值稳定" | 3.4 |
| 分布式训练的 all-reduce 求和顺序不同 → 结果不可复现 | 浮点加法不满足结合律；归约树结构决定结果 | 3.5 陷阱 2 |

**反向的启发**：CS336 里"为什么 bf16 比 fp16 稳"的答案就在第 2 章——bf16 与 fp32 指数位相同（8 位），动态范围一样，只是尾数从 23 位降到 7 位；fp16 指数只有 5 位，范围窄。**范围 vs 分辨率**是两个独立的取舍，这是浮点格式设计的核心。

## 25.5 一张"学完能做什么"的清单

- 拿到一个矩阵，能先问"它什么结构、多大、条件数多少"，再选算法（见配套速查文档的决策树）；
- 看到一段数值代码，能指出它可能在哪一步丢位（抵消、上溢、不正交、不收敛）；
- 能读懂 LAPACK/ARPACK 文档里"backward stable""requires $A \succ 0$"这些条件，并估出算法在给定规模下的时间与内存量级。

---

# 第 26 章 复习清单与自测题

## 26.1 复习清单（能不看笔记答"是"才算过）

| # | 检查项 | 章节 |
|---|---|---|
| 1 | 能写出 $\epsilon_{\text{machine}} = 2^{-53}$ 并解释它和 `finfo.eps` 差 2 倍的原因 | 2.1 |
| 2 | 能背出浮点运算模型 $x \circledast y = (x * y)(1+\epsilon)$，并用它推出 $n\epsilon\sum\lvert x_i\rvert$ | 3.3 |
| 3 | 能解释为什么顺序求和是向后稳定的、却不是前向稳定的 | 3.3 |
| 4 | 能完整证明有限维范数等价（四步，含紧性） | 4.2 |
| 5 | 能用 $\kappa$ 估算"丢几位"，并解释为什么不能看 $\det$ | 5.3–5.4 |
| 6 | 能写出 Newton 的 $\delta_{n+1} = \delta_n^2/2$ 并说明"位数翻倍" | 6.3 |
| 7 | 能解释 CGS 与 MGS 的唯一差别，以及为什么 MGS 的正交性误差是 $\kappa\epsilon$ 而非 $\kappa^2\epsilon$ | 7.3 |
| 8 | 能写出 Householder 的 $v = \operatorname{sign}(x_1)\lVert x\rVert e_1 + x$ 并说明符号的作用 | 8.2 |
| 9 | 能算 Householder QR（$2mn^2 - \frac{2}{3}n^3$）与 Givens QR（$3mn^2 - n^3$）的比值 | 8.4 |

| # | 检查项 | 章节 |
|---|---|---|
| 10 | 能解释为什么部分主元不改善条件数、却能救那个 $2\times 2$ 反例 | 9.3–9.5 |
| 11 | 能说出 Cholesky 三个"免费好处" | 10.1 |
| 12 | 能用 Eckart–Young 解释"低秩逼近误差 = 尾巴奇异值" | 12.3 |
| 13 | 能解释特征值条件数 $1/\lvert y^*x\rvert$ 与 $\kappa(A)$ 的区别 | 13.3 |
| 14 | 能写出幂法误差的两个指数（$k$ 与 $2k$）并说明原因 | 14.3 |
| 15 | 能说明 QR 迭代 = 同时幂法 = 同时反迭代 | 15.2 |
| 16 | 能解释为什么朴素重启失败（最后一行不是 $e_k^*$ 的倍数） | 19.3 |
| 17 | 能说出 CSC 的三个数组与内存量级 | 20.2 |
| 18 | 能解释缓存无关矩阵乘为什么达到 $\Theta(n^3/\sqrt{Z})$ 级（讲义写法） | 21.2 |
| 19 | 能写出 Cooley–Tukey 的指标重排与三层结构 | 22.2 |
| 20 | 能解释梯形公式在周期函数上为何指数收敛（$I_N = I + \pi\sum a_{2mN}$） | 23.3 |

## 26.2 自测题（10 题，答案在后）

1. 双精度下 $\kappa(A)=10^{12}$，用 LU 解 $Ax=b$，大约能剩几位十进制有效数字？如果改用正规方程解最小二乘，还剩几位？
2. 为什么 `sum([1e16, 1.0, -1e16])` 在浮点下等于 0 而不是 1？给出一个能算对的写法。
3. 一个 $1000\times 1000$ 的矩阵，$\|A\|_2=1$、$\sigma_{\min}=10^{-14}$。它可逆吗？在双精度下 `inv(A)` 会给出什么？
4. 用 Householder QR 与用 CGS 分别解同一个最小二乘问题，残差都约为 $10^{-16}$，但解相差很大。哪个解可信？为什么？
5. 为什么 Cholesky 不需要选主元，而 LU 需要？给出一个"对称但不正定"的反例说明 Cholesky 会失败。
6. 幂法在第 10 步的特征值误差是 $10^{-2}$，第 20 步大约是 $10^{-4}$。估计 $\lvert\lambda_2/\lambda_1\rvert$。
7. 稀疏 Cholesky 在 3D 网格上是 $O(N^2)$，2D 是 $O(N^{3/2})$。为什么 3D 更糟？这对算法选择意味着什么？
8. GMRES(20) 在某个对称正定系统上需要 2000 步，而完整 GMRES 只要 80 步。给出两个可能的解释与各自的验证方法。
9. FFT 的 $O(N\log N)$ 是不是下界？课程材料怎么说的？
10. 训练一个 Transformer 时把参数从 fp32 换成 bf16，loss 曲线出现尖峰；换成 fp16 后直接 NaN。用 18.335 的语言解释这两个现象。

## 26.3 答案

1. 双精度约 16 位；$\kappa = 10^{12}$ 时 LU 剩约 $16 - 12 = 4$ 位。正规方程把 $\kappa$ 平方到 $10^{24}$，**一位都不剩**（完全无意义）。〔对应 5.2、11.3〕

2. $10^{16} + 1$ 在双精度下舍入为 $10^{16}$（1 落在 ulp 之外），再减 $10^{16}$ 得 0。正确写法：先排序（小的先加）、或用 `math.fsum`、或用 Kahan 补偿。〔3.3、3.4〕

3. 数学上可逆（$\sigma_{\min} > 0$），但 $\kappa = 10^{14}$，双精度只剩约 2 位；`inv(A)` 会返回一个"看起来正常"的矩阵，但 $A\,\mathbf{inv}(A)$ 与 $I$ 的偏差可达 $10^{-2}$ 量级。**不要用"能否求逆"判断可解性**。〔5.2、5.3〕

4. 用 Householder 的解可信。CGS 的残差小只说明 $A \approx QR$（向后稳定），但 $Q$ 可能严重不正交，投影方向全错；残差是"在错误子空间里的最优"，不是真解。〔7.3〕

5. 正定性保证消元过程中对角元恒正且 $|l_{ij}| \le 1$，不会增长。反例 $A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ 对称、对角元全正，但特征值为 $-1, 3$，Cholesky 在第二步要开方负数而失败。〔10.1、10.3〕

6. 特征值误差按 $(\lambda_2/\lambda_1)^{2k}$ 收敛：10 步到 $10^{-2}$，20 步到 $10^{-4}$，比值平方为 $10^{-2}$ 每 10 步 $\Rightarrow$ $(\lambda_2/\lambda_1)^{20} = 10^{-2} \Rightarrow \lambda_2/\lambda_1 = 10^{-0.1} \approx 0.79$。〔14.3〕

7. 3D 的分隔集是二维面（$N^{2/3}$ 个点），递归后填充累计到 $O(N^{4/3})$、flop 到 $O(N^2)$；2D 的分隔集是一维线（$N^{1/2}$），所以是 $O(N \log N)$ 填充、$O(N^{3/2})$ flop。含义：3D 大规模问题必须用迭代法 + 多重网格预条件，不能指望稀疏直接法。〔20.5〕

8. (a) 重启长度太小（20 远小于收敛所需的方向数），验证：把 restart 提到 100/200 看迭代数是否骤降；(b) 预条件子太弱，验证：换成 IC(0)/AMG 后看迭代数。注意"对称正定"并不保证 GMRES 快——它保证的是 CG 快。〔17.4〕

9. 不是。讲义明确写"the proof of this remains an open problem"——$\Theta(N \log N)$ 的下界尚未被证明，只是没有更快的已知算法（在精确算术模型下）。〔22.4〕

10. bf16 与 fp32 指数位相同（8 位），范围够，但尾数只有 7 位 $\Rightarrow$ 舍入相对误差约 $2^{-8} \approx 4 \times 10^{-3}$，累积成梯度噪声，表现为 loss 尖峰（用 fp32 master weights + 损失缩放缓解）。fp16 指数只有 5 位，最大约 $6.5 \times 10^4$，logits 或梯度一旦超过就变 `inf`，`inf-inf` 变 `NaN`。这正是"范围与分辨率是两个独立维度"的工程版本。〔2.2、25.4〕

---

# 第 27 章 参考来源与访问记录

## 27.1 抓取与核验方式

- 抓取时间：2026-11（下表中每个 URL 均在该时间段访问并核验 HTTP 200）。
- 方法：`curl -sL` 取页面 HTML → 解析 lecture-notes 页的 27 个 PDF 链接 → 逐份下载 → `pdftotext -layout` 转文本 → 逐份阅读。syllabus 与 calendar 页单独抓取并提取正文。

- 失败的链接（如实记录）：`/pages/assignments/` 返回 **404 Page Not Found**；`/pages/exams/`、`/pages/problem-sets/` 均 **301 重定向**（即无对应内容）。课程导航中不存在 Assignments / Exams 栏目。因此本笔记不含任何题号、题面、分数构成以外的评分细节。

## 27.2 课程页面

| 页面 | URL |
| --- | --- |
| 课程主页 | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/ |
| Lecture Notes | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/pages/lecture-notes/ |
| Syllabus | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/pages/syllabus/ |
| Calendar | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/pages/calendar/ |
| Resource Index | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/pages/resource-index/ |

## 27.3 讲义 PDF（27 份，均已下载并转文本阅读）

| 材料 | URL |
| --- | --- |
| Lec 10：Householder Reflectors and Givens Rotations | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/24dd28c58f71fa98f333597707ff8684_MIT18_335JS19_lec10.pdf |
| Lec 11：Performance Experiments with Matrix Multiplication | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/90a515a9caf1cbbe706548d90e659d5a_MIT18_335JS19_lec11.pdf |
| Lec 12：Experiments with Cache-Oblivious Matrix Multiplication | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/94f02a073e2e6c1527e24c71ba8741f4_MIT18_335JS19_lec12.pdf |
| Lec 15：Hessenberg/Tridiagonal Reduction | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/fc0209dbe68002972f36f172077fb976_MIT18_335JS19_lec15.pdf |
| Lec 19：Why restarting Arnoldi/Lanczos is not trivial | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/fa98fed43acb3e2311f6d9328ca31d04_MIT18_335JS19_lec19.pdf |

| 材料 | URL |
| --- | --- |
| Lec 1：Square roots via Newton's method | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/0a734ecc94b60a26213488e68588bc8d_MIT18_335JS19_lec1.pdf |
| Lec 24：Sparse Matrix Algorithms | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/bcd746dca8cbf0d5a890f01a72d8ef64_MIT18_335JS19_lec24.pdf |
| Lec 25：A Brief Overview of Optimization Problems | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/b9aa5a3948b1d993b8e62cff9fd1bdf2_MIT18_335JS19_lec25.pdf |
| Lec 26：Adjoint Methods | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/35b30d8c4896f95264ee1b135fe031f2_MIT18_335JS19_lec26.pdf |
| Lec 27：Adjoint methods and sensitivity analysis for recurrence relations | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/c34c9784410f2cef610f73af72883427_MIT18_335JS19_lec27.pdf |
| Lec 2a：Some myths about floating-point arithmetic | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/881299033165275f3b15ae8b172e6de8_MIT18_335JS19_lec2_supp.pdf |
| Lec 2：Floating-Point Arithmetic, the IEEE Standard | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/2f313023ae3404bc217a81a31b227170_MIT18_335JS19_lec2.pdf |
| Lec 30：Quasi-Newton optimization | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/1b52b607c223977b35ba1d826e4d8df1_MIT18_335JS19_lec30.pdf |
| Lec 33a：Numerical integration and the redemption of the trapezoidal rule | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/4fc9cea286982357dc8f1a64a1b035cf_MIT18_335JS19_lec33_1.pdf |
| Lec 33b：Fourier Cosine Series Examples | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/173a70c0f9676f9166c0baa9c2099750_MIT18_335JS19_lec33_2.pdf |
| Lec 38：Fast Fourier Transform Algorithms | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/edf5b798bef2f1cfe687a6bfcd32a03a_MIT18_335JS19_lec38.pdf |
| Lec 39：Fast Fourier Transform and Fast Fourier Transform in the West | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/f4e0fe09545fcf1a9fa8928385fcf412_MIT18_335JS19_lec39.pdf |
| Lec 3a：Notes on the accuracy of naive summation | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/906cd44ea38d0bb54e13138efd67f3a1_MIT18_335JS19_lec3-1.pdf |

| 材料 | URL |
| --- | --- |
| Lec 3b：Backwards stability of recursive summation | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/ac7d12a336af115e9f4a57740021c8f0_MIT18_335JS19_lec3-2.pdf |
| Lec 4：Notes on the equivalence of norms | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/bcea047068e46ab87f398bdd4209c7f5_MIT18_335JS19_lec4.pdf |
| Lec 6：Modern differential equations solver software | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/469d460f74800c8a1135946a64dee01f_MIT18_335JS19_lec6.pdf |
| Gram-Schmidt Orthogonalization | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/be0cdadd9de56ff8d20d9a0c6d6d9206_MIT18_335JS19_lec9_reading.pdf |
| Ideal-Cache Terminology | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/36335ea8264fbf78790a08b710fdf280_ideal-cache.pdf |
| Introduction to Julia | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/ff680e9dd9857804f64e6f3bf523f80e_Julia-intro.pdf |
| Julia & IJulia Cheat-Sheet | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/fb66800c466efd524aa5d76ddf928bf5_Julia-cheatsheet.pdf |
| The QR Algorithm I | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/f1ce20d2c382d61127d8948b42dd0c86_MIT18_335JS19_lec16_reading.pdf |
| The QR Algorithm II | https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-spring-2019/26ed98a9d8ea1b69ae3bd2817a0bb1c1_MIT18_335JS19_lec16_reading2.pdf |
| ## 27.4 教材与参考书 | |

| 文献 | 说明 |
| --- | --- |
| Trefethen, L. N. & Bau, D. III, *Numerical Linear Algebra*, SIAM, 1997, ISBN 9780898713619 | 课程指定教材；本笔记中标 〔教材〕 的结论出自该书对应主题各讲 |
| Heath, M. T., *Scientific Computing: An Introductory Survey*, SIAM | 课程大纲列出的参考书；本笔记用于交叉核对 flop 计数与算法分类 |
| Barrett, R. et al., *Templates for the Solution of Linear Systems*, SIAM, 1987, ISBN 9780898713282 | 迭代法（CG/GMRES/BiCG）的实践指南，大纲列为 additional reading |
| Bai, Z., Demmel, J. et al., *Templates for the Solution of Algebraic Eigenvalue Problems*, SIAM, ISBN 9780898714715 | 特征值问题的实践指南，大纲列为 additional reading |
| Higham, N. J., "The accuracy of floating point summation," *SIAM J. Sci. Comput.* 14, 783–799 (1993) | Lec 3a 引用的求和误差综述；成对求和 $O(\log n)$ 界出自此文 |
| Kahan, W., "How Java's Floating-Point Hurts Everyone Everywhere" (2004) | Lec 2a 的"浮点迷思"清单改编自该报告 |

## 27.5 本笔记的可靠性分级

| 内容类型 | 可信度 | 识别方式 |
| --- | --- | --- |
| 讲义原文的数字、公式、算法 | 高（逐字核对 PDF 文本） | 段末标 `〔讲义 Lec N〕` |
| 教材标准结论 | 高（数学事实，但未逐页核对页号） | 标 `〔教材〕` |
| 笔者实测数字 | 高（可复现，环境 NumPy 2.3.5 / Python 3.12） | 标 `〔实验〕` |
| 工程经验、外推 | 中（可能有反例） | 标 `〔推断〕` |
| 讲次主题 | 高（来自 calendar 页原文） | 第 1.4 节表格 |

**最不确定的三处**（诚实披露）：

1. **无 PDF 讲次的内容归属**：第 5、7、9、10、11、12、13、17、18 章的部分结论来自教材与标准文献，而非该讲幻灯片；具体措辞、例子、强调重点可能与 Johnson 教授课上所讲不同。
2. **SVD 的 flop 常数**：第 12.4 节只给 $O(mn^2)$ 与"约为 QR 的 2 倍"的量级判断，未给精确常数；不同文献的计数（是否含双对角化、是否含奇异向量累积）差异较大。
3. **"性能差异倍数"类断言**：如 NUMA 影响、循环顺序差 5–10 倍、排序带来 10–100 倍差异，均标为 `〔推断〕`，依赖具体硬件与实现，不应作为定量依据。

---

> **文档结束**。配套实战文档见同目录 `18335_数值方法_项目上手速查.md`（决策树、复杂度表、代码骨架、陷阱清单）。